

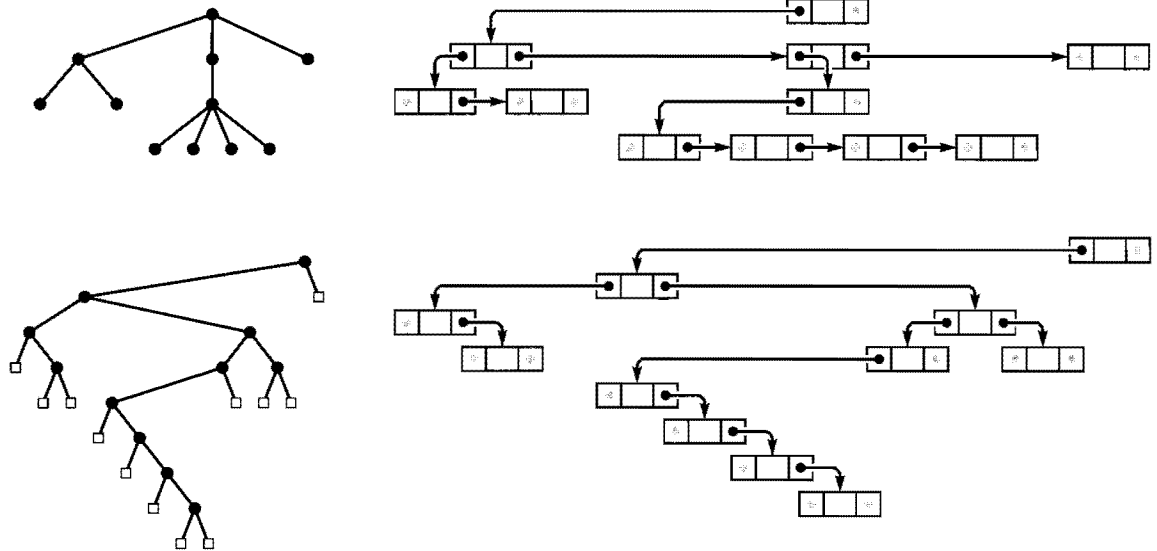


Figure 5.22
Tree representation

Representing an ordered tree by keeping a linked list of the children of each node is equivalent to representing it as a binary tree. The diagram on the right at the top shows a linked-list-of-children representation of the tree on the left at the top, with the list implemented in the right links of nodes, and each node's left link pointing to the first node in the linked list of its children. The diagram on the right at the bottom shows a slightly rearranged version of the diagram above it, and clearly represents the binary tree at the left on the bottom. That is, we can consider the binary tree as representing the tree.

to hold the links to the node's children. Figure 5.22 is an example of such a representation. From this example, it is clear that each node then contains two links, one for the linked list connecting it to its siblings, the other for the linked list of its children.

Property 5.4 *There is a one-to-one correspondence between binary trees and ordered forests.*

The correspondence is depicted in Figure 5.22. We can represent any forest as a binary tree by making the left link of each node point to its leftmost child, and the right link of each node point to its sibling on the right. ■

Definition 5.4 *A rooted tree (or unordered tree) is a node (called the root) connected to a multiset of rooted trees. (Such a multiset is called an unordered forest.)*

The trees that we encountered in Chapter 1 for the connectivity problem are unordered trees. Such trees may be defined as ordered trees where the order in which the children of a node are considered is not significant. We could also choose to define unordered trees as comprising a set of parent-child relationships among nodes. This choice would seem to have little relation to the recursive structures

that we are considering, but it is perhaps the concrete representation that is most true to the abstract notion.

We could choose to represent an unordered tree in a computer with an ordered tree, recognizing that many different ordered trees might represent the same unordered tree. Indeed, the converse problem of determining whether or not two different ordered trees represent the same unordered tree (the *tree-isomorphism* problem) is a difficult one to solve.

The most general type of tree is one where no root node is distinguished. For example, the spanning trees resulting from the connectivity algorithms in Chapter 1 have this property. To define properly *unrooted*, *unordered trees*, or *free trees*, we start with a definition for *graphs*.

Definition 5.5 A *graph* is a set of nodes together with a set of edges that connect pairs of distinct nodes (with at most one edge connecting any pair of nodes).

We can envision starting at some node and following an edge to the constituent node for the edge, then following an edge from that node to another node, and so on. A sequence of edges leading from one node to another in this way with no node appearing twice is called a *simple path*. A graph is *connected* if there is a simple path connecting any pair of nodes. A path that is simple except that the first and final nodes are the same is called a *cycle*.

Every tree is a graph; which graphs are trees? We consider a graph to be a tree if it satisfies any of the following four conditions:

- G has $N - 1$ edges and no cycles.
- G has $N - 1$ edges and is connected.
- Exactly one simple path connects each pair of vertices in G .
- G is connected, but does not remain connected if any edge is removed.

Any one of these conditions is necessary and sufficient to prove the other three. Formally, we should choose one of them to serve as a definition of a *free tree*; informally, we let them collectively serve as the definition.

We represent a free tree simply as a collection of edges. If we choose to represent a free tree as an unordered, ordered or even a binary tree, we need to recognize that, in general, there are many different ways to represent each free tree.

The tree abstraction arises frequently, and the distinctions discussed in this section are important, because knowing different tree abstractions is often an essential ingredient in finding an efficient algorithm and corresponding data structure for a given problem. We often work directly with concrete representations of trees without regard to a particular abstraction, but we also often profit from working with the proper tree abstraction, then considering various concrete representations. We shall see numerous examples of this process throughout the book.

Before moving back to algorithms and implementations, we consider a number of basic mathematical properties of trees; these properties will be of use to us in the design and analysis of tree algorithms.

Exercises

- ▷ 5.56 Give representations of the free tree in Figure 5.20 as a rooted tree and as a binary tree.
- 5.57 How many different ways are there to represent the free tree in Figure 5.20 as an ordered tree?
- ▷ 5.58 Draw three ordered trees that are isomorphic to the ordered tree in Figure 5.20. That is, you should be able to transform the four trees to one another by exchanging children.
- 5.59 Assume that trees contain items for which `eq` is defined. Write a recursive program that deletes all the leaves in a binary tree with items equal to a given item (see Program 5.5).
- 5.60 Change the divide-and-conquer function for finding the maximum item in an array (Program 5.6) to divide the array into k parts that differ by at most 1 in size, recursively find the maximum in each part, and return the maximum of the maxima.
- 5.61 Draw the 3-ary and 4-ary trees corresponding to using $k = 3$ and $k = 4$ in the recursive construction suggested in Exercise 5.60, for an array of 11 elements (see Figure 5.6).
- 5.62 Binary trees are equivalent to binary strings that have one more 0 bit than 1 bit, with the additional constraint that, at any position k , the number of 0 bits that appear strictly to the left of k is no larger than the number of 1 bits strictly to the left of k . A binary tree is either a 0 or two such strings

concatenated together, preceded by a 1. Draw the binary tree that corresponds to the string

1 1 1 0 0 1 0 1 1 0 0 0 1 0 1 1 0 0 0.

- 5.63 Ordered trees are equivalent to balanced strings of parentheses: An ordered tree either is null or is a sequence of ordered trees enclosed in parentheses. Draw the ordered tree that corresponds to the string

((() (() ()) ()) (() () ())) .

- 5.64 Write a program to determine whether or not two arrays of N integers between 0 and $N - 1$ represent isomorphic unordered trees, when interpreted (as in Chapter 1) as parent-child links in a tree with nodes numbered between 0 and $N - 1$. That is, your program should determine whether or not there is a way to renumber the nodes in one tree such that the array representation of the one tree is identical to the array representation of the other tree.
- 5.65 Write a program to determine whether or not two binary trees represent isomorphic unordered trees.
- ▷ 5.66 Draw all the ordered trees that could represent the tree defined by the set of edges 0-1, 1-2, 1-3, 1-4, 4-5.
- 5.67 Prove that, if a connected graph of N nodes has the property that removing any edge disconnects the graph, then the graph has $N - 1$ edges and no cycles.

5.5 Mathematical Properties of Binary Trees

Before beginning to consider tree-processing algorithms, we continue in a mathematical vein by considering a number of basic properties of trees. We focus on binary trees, because we use them frequently throughout this book. Understanding their basic properties will lay the groundwork for understanding the performance characteristics of various algorithms that we will encounter—not only of those that use binary trees as explicit data structures, but also of divide-and-conquer recursive algorithms and other similar applications.

Property 5.5 *A binary tree with N internal nodes has $N + 1$ external nodes.*

We prove this property by induction: A binary tree with no internal nodes has one external node, so the property holds for $N = 0$. For $N > 0$, any binary tree with N internal nodes has k internal nodes in its left subtree and $N - 1 - k$ internal nodes in its right subtree for

some k between 0 and $N - 1$, since the root is an internal node. By the inductive hypothesis, the left subtree has $k + 1$ external nodes and the right subtree has $N - k$ external nodes, for a total of $N + 1$. ■

Property 5.6 *A binary tree with N internal nodes has $2N$ links: $N - 1$ links to internal nodes and $N + 1$ links to external nodes.*

In any rooted tree, each node, except the root, has a unique parent, and every edge connects a node to its parent, so there are $N - 1$ links connecting internal nodes. Similarly, each of the $N + 1$ external nodes has one link, to its unique parent. ■

The performance characteristics of many algorithms depend not just on the number of nodes in associated trees, but on various structural properties.

Definition 5.6 *The level of a node in a tree is one higher than the level of its parent (with the root at level 0). The height of a tree is the maximum of the levels of the tree's nodes. The path length of a tree is the sum of the levels of all the tree's nodes. The internal path length of a binary tree is the sum of the levels of all the tree's internal nodes. The external path length of a binary tree is the sum of the levels of all the tree's external nodes.*

A convenient way to compute the path length of a tree is to sum, for all k , the product of k and the number of nodes at level k .

These quantities also have simple recursive definitions that follow directly from the recursive definitions of trees and binary trees. For example, the height of a tree is 1 greater than the maximum of the height of the subtrees of its root, and the path length of a tree with N nodes is the sum of the path lengths of the subtrees of its root plus $N - 1$. The quantities also relate directly to the analysis of recursive algorithms. For example, for many recursive computations, the height of the corresponding tree is precisely the maximum depth of the recursion, or the size of the stack needed to support the computation.

Property 5.7 *The external path length of any binary tree with N internal nodes is $2N$ greater than the internal path length.*

We could prove this property by induction, but an alternate proof (which also works for Property 5.6) is instructive. Observe that any binary tree can be constructed by the following process: Start with the

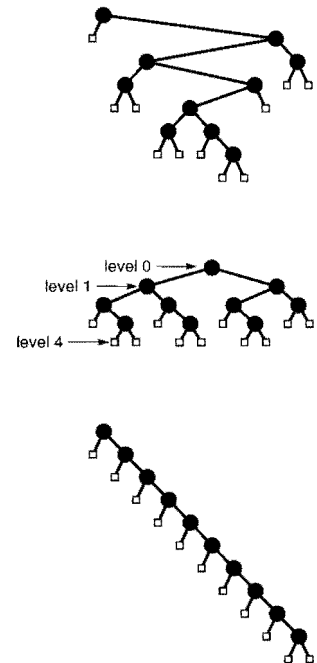


Figure 5.23
Three binary trees with 10 internal nodes

The binary tree shown at the top has height 7, internal path length 31 and external path length 51. A fully balanced binary tree (center) with 10 internal nodes has height 4, internal path length 19 and external path length 39 (no binary tree with 10 nodes has smaller values for any of these quantities). A degenerate binary tree (bottom) with 10 internal nodes has height 10, internal path length 45 and external path length 65 (no binary tree with 10 nodes has larger values for any of these quantities).

binary tree consisting of one external node. Then, repeat the following N times: Pick an external node and replace it by a new internal node with two external nodes as children. If the external node chosen is at level k , the internal path length is increased by k , but the external path length is increased by $k + 2$ (one external node at level k is removed, but two at level $k + 1$ are added). The process starts with a tree with internal and external path lengths both 0 and, for each of N steps, increases the external path length by 2 more than the internal path length. ■

Property 5.8 *The height of a binary tree with N internal nodes is at least $\lg N$ and at most $N - 1$.*

The worst case is a degenerate tree with only one leaf, with $N - 1$ links from the root to the leaf (see Figure 5.23). The best case is a balanced tree with 2^i internal nodes at every level i except the bottom level (see Figure 5.23). If the height is h , then we must have

$$2^{h-1} < N + 1 \leq 2^h,$$

since there are $N + 1$ external nodes. This inequality implies the property stated: The best-case height is precisely equal to $\lg N$ rounded up to the nearest integer. ■

Property 5.9 *The internal path length of a binary tree with N internal nodes is at least $N \lg(N/4)$ and at most $N(N - 1)/2$.*

The worst case and the best case are achieved for the same trees referred to in the discussion of Property 5.8 and depicted in Figure 5.23. The internal path length of the worst-case tree is $0 + 1 + 2 + \dots + (N - 1) = N(N - 1)/2$. The best case tree has $(N + 1)$ external nodes at height no more than $\lfloor \lg N \rfloor$. Multiplying these and applying Property 5.7, we get the bound $(N + 1)\lfloor \lg N \rfloor - 2N < N \lg(N/4)$. ■

As we shall see, binary trees appear extensively in computer applications, and performance is best when the binary trees are fully balanced (or nearly so). For example, the trees that we use to describe divide-and-conquer algorithms such as binary search and mergesort are fully balanced (see Exercise 5.74). In Chapters 9 and 13, we shall examine explicit data structures that are based on balanced trees.

These basic properties of trees provide the information that we need to develop efficient algorithms for a number of practical problems. More detailed analyses of several of the specific algorithms

that we shall encounter require sophisticated mathematical analysis, although we can often get useful estimates with straightforward inductive arguments like the ones that we have used in this section. We discuss further mathematical properties of trees as needed in the chapters that follow. At this point, we are ready to move back to algorithmic matters.

Exercises

- ▷ 5.68 How many external nodes are there in an M -ary tree with N internal nodes? Use your answer to give the amount of memory required to represent such a tree, assuming that links and items require one word of memory each.
- 5.69 Give upper and lower bounds on the height of an M -ary tree with N internal nodes.
- 5.70 Give upper and lower bounds on the internal path length of an M -ary tree with N internal nodes.
- 5.71 Give upper and lower bounds on the number of leaves in a binary tree with N nodes.
- 5.72 Show that if the levels of the external nodes in a binary tree differ by a constant, then the height is $O(\log N)$.
- 5.73 A *Fibonacci tree* of height $n > 2$ is a binary tree with a Fibonacci tree of height $n - 1$ in one subtree and a Fibonacci tree of height $n - 2$ in the other subtree. A Fibonacci tree of height 0 is a single external node, and a Fibonacci tree of height 1 is a single internal node with two external children (see Figure 5.14). Give the height and external path length of a Fibonacci tree of height n , as a function of N , the number of nodes in the tree.
- 5.74 A *divide-and-conquer tree* of N nodes is a binary tree with a root labeled N , a divide-and-conquer tree of $\lfloor N/2 \rfloor$ nodes in one subtree, and a divide-and-conquer tree of $\lceil N/2 \rceil$ nodes in the other subtree. (Figure 5.6 depicts a divide-and-conquer tree.) Draw divide-and-conquer trees with 11, 15, 16, and 23 nodes.
- 5.75 Prove by induction that the internal path length of a divide-and-conquer tree is between $N \lg N$ and $N \lg N + N$.
- 5.76 A *combine-and-conquer tree* of N nodes is a binary tree with a root labeled N , a combine-and-conquer tree of $\lfloor N/2 \rfloor$ nodes in one subtree, and a combine-and-conquer tree of $\lceil N/2 \rceil$ nodes in the other subtree (see Exercise 5.18). Draw combine-and-conquer trees with 11, 15, 16, and 23 nodes.
- 5.77 Prove by induction that the internal path length of a combine-and-conquer tree is between $N \lg N$ and $N \lg N + N$.
- 5.78 A *complete* binary tree is one with all levels filled, except possibly the final one, which is filled from left to right, as illustrated in Figure 5.24. Prove that the internal path length of a complete tree with N nodes is between $N \lg N$ and $N \lg N + N$.

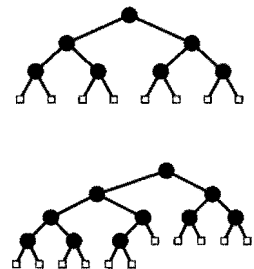


Figure 5.24
Complete binary trees with seven and 10 internal nodes

When the number of external nodes is a power of 2 (top), the external nodes in a complete binary tree are all at the same level. Otherwise (bottom), the external nodes appear on two levels, with the internal nodes to the left of the external nodes on the next-to-bottom level.

5.6 Tree Traversal

Before considering algorithms that construct binary trees and trees, we consider algorithms for the most basic tree-processing function: *tree traversal*: Given a pointer to a tree, we want to process every node in the tree systematically. In a linked list, we move from one node to the next by following the single link; for trees, however, we have decisions to make, because there may be multiple links to follow.

We begin by considering the process for binary trees. For linked lists, we had two basic options (see Program 5.5): process the node and then follow the link (in which case we would visit the nodes in order), or follow the link and then process the node (in which case we would visit the nodes in reverse order). For binary trees, we have two links, and we therefore have three basic orders in which we might visit the nodes:

- *Preorder*, where we visit the node, then visit the left and right subtrees
- *Inorder*, where we visit the left subtree, then visit the node, then visit the right subtree
- *Postorder*, where we visit the left and right subtrees, then visit the node

We can implement these methods easily with a recursive program, as shown in Program 5.14, which is a direct generalization of the linked-list-traversal program in Program 5.5. To implement traversals in the other orders, we permute the function calls in Program 5.14 in the appropriate manner. Figure 5.26 shows the order in which we visit the nodes in a sample tree for each order. Figure 5.25 shows the sequence of function calls that is executed when we invoke Program 5.14 on the sample tree in Figure 5.26.

We have already encountered the same basic recursive processes on which the different tree-traversal methods are based, in divide-and-conquer recursive programs (see Figures 5.8 and 5.11), and in arithmetic expressions. For example, doing preorder traversal corresponds to drawing the marks on the ruler first, then making the recursive calls (see Figure 5.11); doing inorder traversal corresponds to moving the biggest disk in the towers of Hanoi solution in between recursive calls that move all of the others; doing postorder traversal corresponds to evaluating postfix expressions, and so forth. These correspondences

Program 5.14 Recursive tree traversal

This recursive function takes a link to a tree as an argument and calls the function `visit` with each of the nodes in the tree as argument. As is, the function implements a preorder traversal; if we move the call to `visit` between the recursive calls, we have an inorder traversal; and if we move the call to `visit` after the recursive calls, we have a postorder traversal.

```
void traverse(link h, void (*visit)(link))
{
    if (h == NULL) return;
    (*visit)(h);
    traverse(h->l, visit);
    traverse(h->r, visit);
}
```

give us immediate insight into the mechanisms behind tree traversal. For example, we know that every other node in an inorder traversal is an external node, for the same reason that every other move in the towers of Hanoi problem involves the small disk.

It is also useful to consider nonrecursive implementations that use an explicit pushdown stack. For simplicity, we begin by considering an abstract stack that can hold items or trees, initialized with the tree to be traversed. Then, we enter into a loop, where we pop and process the top entry on the stack, continuing until the stack is empty. If the popped entity is an item, we visit it; if the popped entity is a tree, then we perform a sequence of push operations that depends on the desired ordering:

- For *preorder*, we push the right subtree, then the left subtree, and then the node.
- For *inorder*, we push the right subtree, then the node, and then the left subtree.
- For *postorder*, we push the node, then the right subtree, and then the left subtree.

We do not push null trees onto the stack. Figure 5.27 shows the stack contents as we use each of these three methods to traverse the sample tree in Figure 5.26. We can easily verify by induction that this method produces the same output as the recursive one for any binary tree.

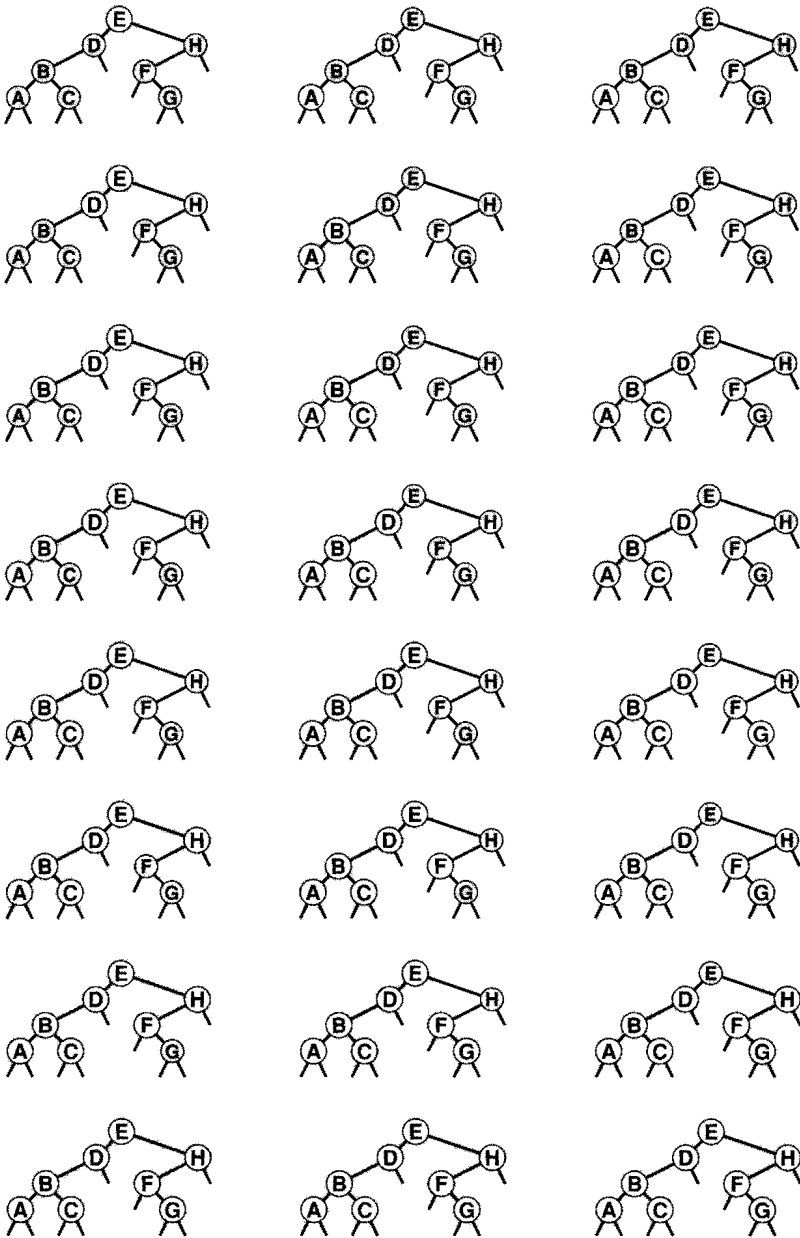
```
traverse E
  visit E
  traverse D
    visit D
    traverse B
      visit B
      traverse A
        visit A
        traverse *
        traverse *
      traverse C
        visit C
        traverse *
        traverse *
    traverse *
  traverse H
    visit H
    traverse F
      visit F
      traverse *
      traverse G
        visit G
        traverse *
        traverse *
    traverse *
```

Figure 5.25
Preorder-traversal function calls

This sequence of function calls constitutes preorder traversal for the example tree in Figure 5.26.

Figure 5.26
Tree-traversal orders

These sequences indicate the order in which we visit nodes for pre-order (left), inorder (center), and postorder (right) tree traversal.



Program 5.15 Preorder traversal (nonrecursive)

This nonrecursive stack-based function is functionally equivalent to its recursive counterpart, Program 5.14.

```
void traverse(link h, void (*visit)(link))
{
    STACKinit(max); STACKpush(h);
    while (!STACKempty())
    {
        (*visit)(h = STACKpop());
        if (h->r != NULL) STACKpush(h->r);
        if (h->l != NULL) STACKpush(h->l);
    }
}
```

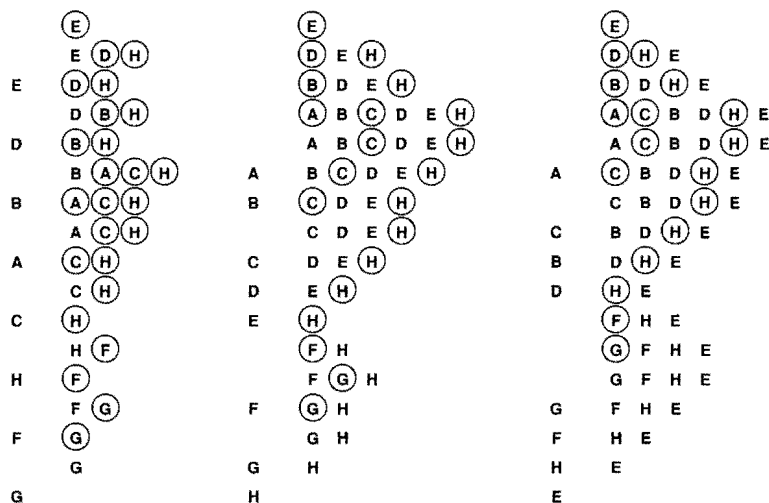
The scheme described in the previous paragraph is a conceptual one that encompasses the three traversal methods, but the implementations that we use in practice are slightly simpler. For example, for preorder, we do not need to push nodes onto the stack (we visit the root of each tree that we pop), and we therefore can use a simple stack that contains only one type of item (tree link), as in the nonrecursive implementation in Program 5.15. The system stack that supports the recursive program contains return addresses and argument values, rather than items or nodes, but the actual sequence in which we do the computations (visit the nodes) is the same for the recursive and the stack-based methods.

A fourth natural traversal strategy is simply to visit the nodes in a tree as they appear on the page, reading down from top to bottom and from left to right. This method is called *level-order* traversal because all the nodes on each level appear together, in order. Figure 5.28 shows how the nodes of the tree in Figure 5.26 are visited in level order.

Remarkably, we can achieve level-order traversal by substituting a queue for the stack in Program 5.15, as shown in Program 5.16. For preorder, we use a LIFO data structure; for level order, we use a FIFO data structure. These programs merit careful study, because they represent approaches to organizing work remaining to be done that differ in an essential way. In particular, level order does *not* correspond

Figure 5.27
Stack contents for tree-
traversal algorithms

These sequences indicate the stack contents for preorder (left), inorder (center), and postorder (right) tree traversal (see Figure 5.26), for an idealized model of the computation, similar to the one that we used in Figure 5.5, where we put the item and its two subtrees on the stack, in the indicated order.

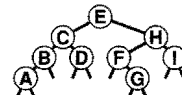
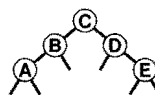


to a recursive implementation relates to the recursive structure of the tree.

Preorder, postorder, and level order are well defined for forests as well. To make the definitions consistent, think of a forest as a tree with an imaginary root. Then, the preorder rule is “visit the root, then visit each of the subtrees,” the postorder rule is “visit each of the subtrees, then visit the root.” The level-order rule is the same as for binary trees. Direct implementations of these methods are straightforward generalizations of the stack-based preorder traversal programs (Programs 5.14 and 5.15) and the queue-based level-order traversal program (Program 5.16) for binary trees that we just considered. We omit consideration of implementations because we consider a more general procedure in Section 5.8.

Exercises

- ▷ 5.79 Give preorder, inorder, postorder, and level-order traversals of the following binary trees:



Program 5.16 Level-order traversal

Switching the underlying data structure in preorder traversal (see Program 5.15) from a stack to a queue transforms the traversal into a level-order one.

```
void traverse(link h, void (*visit)(link))
{
    QUEUEinit(max); QUEUEput(h);
    while (!QUEUEempty())
    {
        (*visit)(h = QUEUEget());
        if (h->l != NULL) QUEUEput(h->l);
        if (h->r != NULL) QUEUEput(h->r);
    }
}
```

▷ 5.80 Show the contents of the queue during the level order traversal (Program 5.16) depicted in Figure 5.28, in the style of Figure 5.27.

5.81 Show that preorder for a forest is the same as preorder for the corresponding binary tree (see Property 5.4), and that postorder for a forest is the same as inorder for the binary tree.

○ 5.82 Give a nonrecursive implementation of inorder traversal.

• 5.83 Give a nonrecursive implementation of postorder traversal.

• 5.84 Write a program that takes as input the preorder and inorder traversals of a binary tree, and produces as output the level-order traversal of the tree.

5.7 Recursive Binary-Tree Algorithms

The tree-traversal algorithms that we considered in Section 5.6 exemplify the basic fact that we are led to consider recursive algorithms for binary trees, because of these trees' very nature as recursive structures. Many tasks admit direct recursive divide-and-conquer algorithms, which essentially generalize the traversal algorithms. We process a tree by processing the root node and (recursively) its subtrees; we can do computation before, between, or after the recursive calls (or possibly all three).

We frequently need to find the values of various structural parameters for a tree, given only a link to the tree. For example, Program 5.17

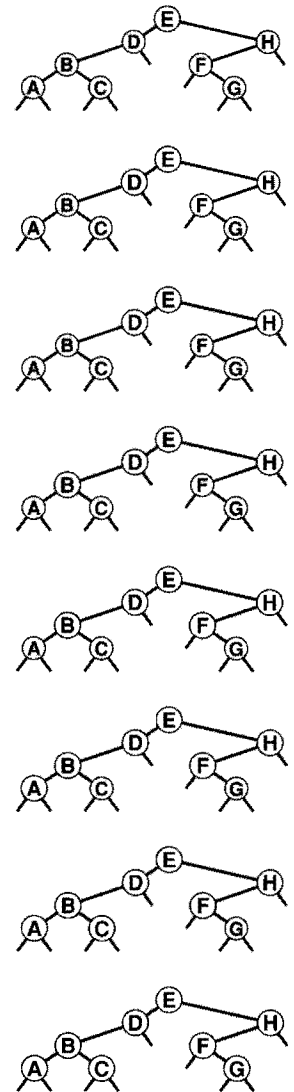


Figure 5.28
Level-order traversal

This sequence depicts the result of visiting nodes in order from top to bottom and left to right in the tree.

Program 5.17 Computation of tree parameters

We can use simple recursive procedures such as these to learn basic structural properties of trees.

```
int count(link h)
{
    if (h == NULL) return 0;
    return count(h->l) + count(h->r) + 1;
}

int height(link h)
{ int u, v;
  if (h == NULL) return -1;
  u = height(h->l); v = height(h->r);
  if (u > v) return u+1; else return v+1;
}
```

comprises recursive functions for computing the number of nodes in and the height of a given tree. The functions follow immediately from Definition 5.6. Neither of these functions depends on the order in which the recursive calls are processed: they process all the nodes in the tree and return the same answer if we, for example, exchange the recursive calls. Not all tree parameters are so easily computed: for example, a program to compute efficiently the internal path length of a binary tree is more challenging (see Exercises 5.88 through 5.90).

Another function that is useful whenever we write programs that process trees is one that prints out or draws the tree. For example, Program 5.18 is a recursive procedure that prints out a tree in the format illustrated in Figure 5.29. We can use the same basic recursive scheme to draw more elaborate representations of trees, such as those that we use in the figures in this book (see Exercise 5.85).

Program 5.18 is an inorder traversal—if we print the item before the recursive calls, we get a preorder traversal, which is also illustrated in Figure 5.29. This format is a familiar one that we might use, for example, for a family tree, or to list files in a tree-based file system, or to make an outline of a printed document. For example, doing a preorder traversal of the tree in Figure 5.19 gives a version of the table of contents of this book.

Program 5.18 Quick tree-print function

This recursive program keeps track of the tree height and uses that information for indentation in printing out a representation of the tree that we can use to debug tree-processing programs (see Figure 5.29). It assumes that items in nodes are characters.

```
void printnode(char c, int h)
{ int i;
  for (i = 0; i < h; i++) printf(" ");
  printf("%c\n", c);
}
void show(link x, int h)
{
  if (x == NULL) { printnode('*', h); return; }
  show(x->r, h+1);
  printnode(x->item, h);
  show(x->l, h+1);
}
```

Our first example of a program that builds an explicit binary tree structure is associated with the find-the-maximum application that we considered in Section 5.2. Our goal is to build a *tournament*: a binary tree where the item in every internal node is a copy of the larger of the items in its two children. In particular, the item at the root is a copy of the largest item in the tournament. The items in the leaves (nodes with no children) constitute the data of interest, and the rest of the tree is a data structure that allows us to find the largest of the items efficiently.

Program 5.19 is a recursive program that builds a tournament from the items in an array. A modification of Program 5.6, it thus uses a divide-and-conquer recursive strategy: To build a tournament for a single item, we create (and return) a leaf containing that item. To build a tournament for $N > 1$ items, we use the divide-and-conquer strategy: Divide the items in half, build tournaments for each half, and create a new node with links to the two tournaments and with an item that is a copy of the larger of the items in the roots of the two tournaments.

Figure 5.30 is an example of an explicit tree structure that might be built by Program 5.19. Building a recursive data structure such

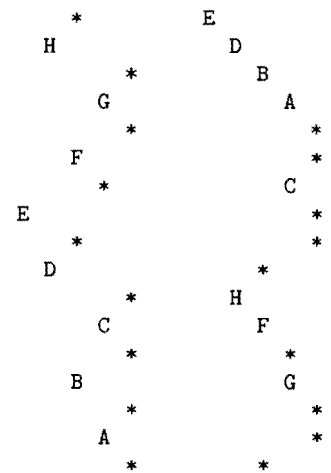


Figure 5.29
Printing a tree (inorder and
preorder)

The output at the left results from using Program 5.18 on the sample tree in Figure 5.26, and exhibits the tree structure in a manner similar to the graphical representation that we have been using, rotated 90 degrees. The output at the right is from the same program with the print statement moved to the beginning; it exhibits the tree structure in a familiar outline format.

Program 5.19 Construction of a tournament

This recursive function divides a file $a[1], \dots, a[r]$ into the two parts $a[1], \dots, a[m]$ and $a[m+1], \dots, a[r]$, builds tournaments for the two parts (recursively), and makes a tournament for the whole file by setting links in a new node to the recursively built tournaments and setting its item value to the larger of the items in the roots of the two recursively built tournaments.

```
typedef struct node *link;
struct node { Item item; link l, r };
link NEW(Item item, link l, link r)
{ link x = malloc(sizeof *x);
  x->item = item; x->l = l; x->r = r;
  return x;
}
link max(Item a[], int l, int r)
{ int m = (l+r)/2; Item u, v;
  link x = NEW(a[m], NULL, NULL);
  if (l == r) return x;
  x->l = max(a, l, m);
  x->r = max(a, m+1, r);
  u = x->l->item; v = x->r->item;
  if (u > v)
    x->item = u; else x->item = v;
  return x;
}
```

as this one is perhaps preferable in some situations to finding the maximum by scanning the data, as we did in Program 5.6, because the tree structure provides us with the flexibility to perform other operations. The very operation that we use to build the tournament is an important example: Given two tournaments, we can combine them into a single tournament in constant time, by creating a new node, making its left link point to one of the tournaments and its right link point to the other, and taking the larger of the two items (at the roots of the two given tournaments) as the largest item in the combined tournament. We also can consider algorithms for adding items, removing items, and performing other operations. We shall not

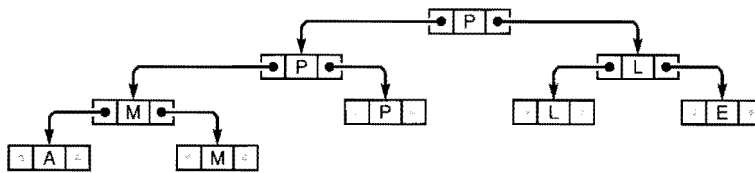


Figure 5.30
Explicit tree for finding the
maximum (tournament)

This figure depicts the explicit tree structure that is constructed by Program 5.19 from the input A M P L E. The data items are in the leaves. Each internal node has a copy of the larger of the items in its two children, so, by induction, the largest item is at the root.

consider such operations in any further detail here because similar data structures with this flexibility are the topic of Chapter 9.

Indeed, tree-based implementations for several of the generalized queue ADTs that we discussed in Section 4.6 are a primary topic of discussion for much of this book. In particular, many of the algorithms in Chapters 12 through 15 are based on *binary search trees*, which are explicit trees that correspond to binary search, in a relationship analogous to the relationship between the explicit structure of Figure 5.30 and the recursive find-the-maximum algorithm (see Figure 5.6). The challenge in implementing and using such structures is to ensure that our algorithms remain efficient after a long sequence of *insert*, *delete*, and other operations.

Our second example of a program that builds a binary tree is a modification of our prefix-expression-evaluation program in Section 5.1 (Program 5.4) to construct a tree representing a prefix expression, instead of just evaluating it (see Figure 5.31). Program 5.20 uses the same recursive scheme as Program 5.4, but the recursive function returns a link to a tree, rather than a value. We create a new tree node for each character in the expression: Nodes corresponding to operators have links to their operands, and the leaf nodes contain the variables (or constants) that are inputs to the expression.

Translation programs such as compilers often use such internal tree representations for programs, because the trees are useful for many purposes. For example, we might imagine operands corresponding to variables that take on values, and we could generate machine code to evaluate the expression represented by the tree with a postorder traversal. Or, we could use the tree to print out the expression in infix with an inorder traversal or in postfix with a postorder traversal.

We considered the few examples in this section to introduce the concept that we can build and process explicit linked tree structures with recursive programs. To do so effectively, we need to consider

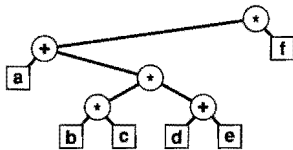


Figure 5.31
Parse tree

This tree is constructed by Program 5.20 for the prefix expression `++a**b c+d e f`. It is a natural way to represent the expression: Each operand is in a leaf (which we draw here as an external node), and each operator is to be applied to the expressions represented by the left and right subtrees of the node containing the operator.

Program 5.20 Construction of a parse tree

Using the same strategy that we used to evaluate prefix expressions (see Program 5.4), this program builds a parse tree from a prefix expression. For simplicity, we assume that operands are single characters. Each call of the recursive function creates a new node with the next character from the input as the token. If the token is an operand, we return the tree built (recursively) for the two arguments.

```

char *a; int i;
typedef struct Tnode* link;
struct Tnode { char token; link l, r; };
link NEW(char token, link l, link r)
{ link x = malloc(sizeof *x);
  x->token = token; x->l = l; x->r = r;
  return x;
}
link parse()
{ char t = a[i++];
  link x = NEW(t, NULL, NULL);
  if ((t == '+') || (t == '*'))
    { x->l = parse(); x->r = parse(); }
  return x;
}

```

the performance of various algorithms, alternate representations, non-recursive alternatives, and many other details. However, we shall defer consideration of tree-processing programs in further detail until Chapter 12, because we use trees primarily for descriptive purposes in Chapters 7 through 11. We return to explicit tree implementations in Chapter 12 because they form the basis of numerous algorithms that we consider in Chapters 12 through 15.

Exercises

- 5.85 Modify Program 5.18 to output a PostScript program that draws the tree, in a format like that used in Figure 5.23, but without the small boxes to

represent the external nodes. Use `moveto` and `lineto` to draw lines, and the user-defined operator

```
/node { newpath moveto currentpoint 4 0 360 arc fill} def
```

to draw nodes. After this definition, the call `node` draws a black dot at the coordinates on the stack (see Section 4.3).

- ▷ 5.86 Write a program that counts the leaves in a binary tree.
- ▷ 5.87 Write a program that counts the number of nodes in a binary tree that have one external and one internal child.
- ▷ 5.88 Write a recursive program that computes the internal path length of a binary tree, using Definition 5.6.
- 5.89 Determine the number of function calls made by your program when it is computing the internal path length of a binary tree. Prove your answer by induction.
- 5.90 Write a recursive program that computes the internal path length of a binary tree in time proportional to the number of nodes in the tree.
- 5.91 Write a recursive program that deletes all the leaves with a given key from a tournament (see Exercise 5.59).

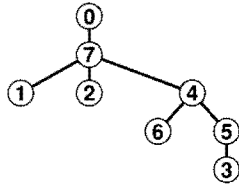
5.8 Graph Traversal

For our final example of a recursive program in this chapter, we consider one of the most important of all recursive programs: recursive graph traversal, or *depth-first search*. This method for systematically visiting all the nodes in a graph is a direct generalization of the tree-traversal methods that we considered in Section 5.6, and it serves as the basis for many basic algorithms for processing graphs (see Part 7). It is a simple recursive algorithm. Starting at any node v , we

- Visit v .
- (Recursively) visit each (*unvisited*) node attached to v .

If the graph is connected, we eventually reach all of the nodes. Program 5.21 is an implementation of this recursive procedure.

For example, suppose that we use the adjacency-list representation depicted in the sample graph in Figure 3.15. Figure 5.32 shows the recursive calls made during the depth-first search of this graph, and the sequence on the left in Figure 5.33 depicts the way in which we follow the edges in the graph. We follow each edge in the graph, with one of two possible outcomes: if the edge takes us to a node that we have already visited, we ignore it; if it takes us to a node that we have



visit 0
 visit 7 (first on 0's list)
 visit 1 (first on 7's list)
 check 7 on 1's list
 check 0 on 1's list
 visit 2 (second on 7's list)
 check 7 on 2's list
 check 0 on 2's list
 check 0 on 7's list
 visit 4 (fourth on 7's list)
 visit 6 (first on 4's list)
 check 4 on 6's list
 check 0 on 6's list
 visit 5 (second on 4's list)
 check 0 on 5's list
 check 4 on 5's list
 visit 3 (third on 5's list)
 check 5 on 3's list
 check 4 on 3's list
 check 7 on 4's list
 check 3 on 4's list
 check 5 on 0's list
 check 2 on 0's list
 check 1 on 0's list
 check 6 on 0's list

Figure 5.32
Depth-first-search function calls

This sequence of function calls constitutes depth-first search for the example graph in Figure 3.15. The tree that depicts the recursive-call structure (top) is called the depth-first-search tree.

Program 5.21 Depth-first search

To visit all the nodes connected to node k in a graph, we mark it as *visited*, then (recursively) visit all the unvisited nodes on k 's adjacency list.

```

void traverse(int k, void (*visit)(int))
{ link t;
  (*visit)(k); visited[k] = 1;
  for (t = adj[k]; t != NULL; t = t->next)
    if (!visited[t->v]) traverse(t->v, visit);
}

```

not yet visited, we follow it there via a recursive call. The set of all edges that we follow in this way forms a spanning tree for the graph.

The difference between depth-first search and general tree traversal (see Program 5.14) is that we need to guard explicitly against visiting nodes that we have already visited. In a tree, we never encounter any such nodes. Indeed, if the graph is a tree, recursive depth-first search starting at the root is equivalent to preorder traversal.

Property 5.10 *Depth-first search requires time proportional to $V + E$ in a graph with V vertices and E edges, using the adjacency lists representation.*

In the adjacency lists representation, there is one list node corresponding to each edge in the graph, and one list head pointer corresponding to each vertex in the graph. Depth-first search touches all of them, at most once. ■

Because it also takes time proportional to $V + E$ to build the adjacency lists representation from an input sequence of edges (see Program 3.19), depth-first search gives us a linear-time solution to the connectivity problem of Chapter 1. For huge graphs, however, the union-find solutions might still be preferable, because representing the whole graph takes space proportional to E , while the union-find solutions take space only proportional to V .

As we did with tree traversal, we can define a graph-traversal method that uses an explicit stack, as depicted in Figure 5.34. We can think of an abstract stack that holds dual entries: a node and a pointer into that node's adjacency list. With the stack initialized

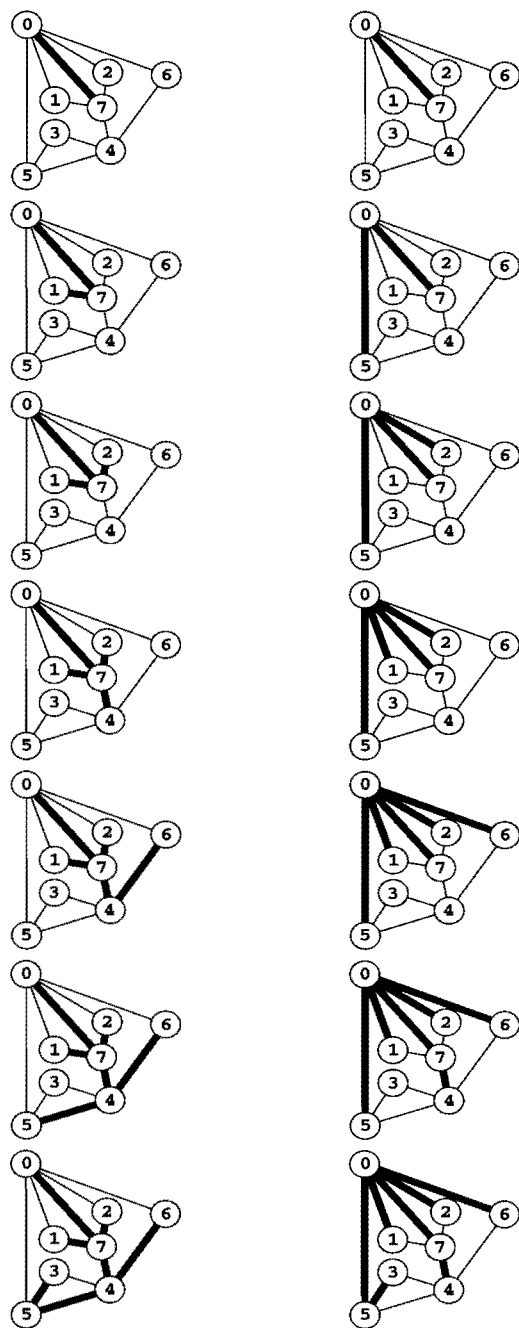
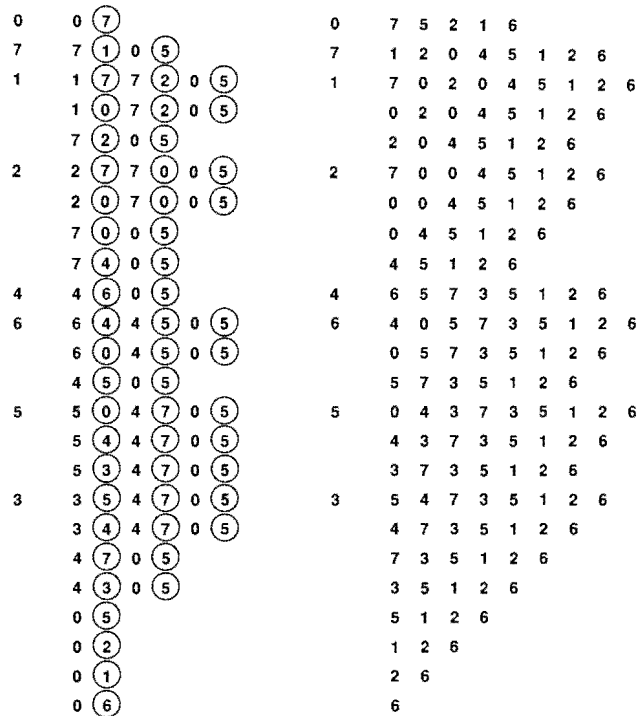


Figure 5.33
Depth-first search and
breadth-first search

Depth-first search (left) moves from node to node, backing up to the previous node to try the next possibility whenever it has tried every possibility at a given node. Breadth-first search (right) exhausts all the possibilities at one node before moving to the next.

Figure 5.34
Depth-first-search stack dynamics

We can think of the pushdown stack supporting depth-first search as containing a node and a reference to that node's adjacency list (indicated by a circled node) (left). Thus, we begin with node 0 on the stack, with reference to the first node on its list, node 7. Each line indicates the result of popping the stack, pushing a reference to the next node on the list for nodes that have been visited, and pushing an entry on the stack for nodes that have not been visited. Alternatively, we can think of the process as simply pushing all nodes adjacent to any unvisited node onto the stack (right).



to the start node and a pointer initialized to the first node on that node's adjacency list, the depth-first search algorithm is equivalent to entering into a loop, where we visit the node at the top of the stack (if it has not already been visited); save the node referenced by the current adjacency-list pointer; update the adjacency list reference to the next node (popping the entry if at the end of the adjacency list); and push a stack entry for the saved node, referencing the first node on its adjacency list.

Alternatively, as we did for tree traversal, we can consider the stack to contain links to nodes only. With the stack initialized to the start node, we enter into a loop where we visit the node at the top of the stack (if it has not already been visited), then push all the nodes adjacent to it onto the stack. Figure 5.34 illustrates that both of these methods are equivalent to depth-first search for our example graph, and the equivalence indeed holds in general.

Program 5.22 Breadth-first search

To visit all the nodes connected to node k in a graph, we put k onto a FIFO queue, then enter into a loop where we get the next node from the queue, and, if it has not been visited, visit it and push all the unvisited nodes on its adjacency list, continuing until the queue is empty.

```
void traverse(int k, void (*visit)(int))
{ link t;
  QUEUEinit(V); QUEUEput(k);
  while (!QUEUEempty())
    if (visited[k = QUEUEget()] == 0)
      {
        (*visit)(k); visited[k] = 1;
        for (t = adj[k]; t != NULL; t = t->next)
          if (visited[t->v] == 0) QUEUEput(t->v);
      }
}
```

The visit-the-top-node-and-push-all-its-neighbors algorithm is a simple formulation of depth-first search, but it is clear from Figure 5.34 that it suffers the disadvantage of possibly leaving multiple copies of each node on the stack. It does so even if we test whether each node that is about to go on the stack has been visited and refrain from putting the node in the stack if it has been. To avoid this problem, we can use a stack implementation that disallows duplicates by using a forget-the-old-item policy, because the copy nearest the top of the stack is always the first one visited, so the others are simply popped.

The stack dynamics for depth-first search that are illustrated in Figure 5.34 depend on the nodes on each adjacency list ending up on the stack in the same order that they appear in the list. To get this ordering for a given adjacency list when pushing one node at a time, we would have to push the last node first, then the next-to-last node, and so forth. Moreover, to limit the stack size to the number of vertices while at the same time visiting the nodes in the same order as in depth-first search, we need to use a stack discipline with a forget-the-old-item policy. If visiting the nodes in the same order as depth-first search is not important to us, we can avoid both of these complications and directly formulate a nonrecursive stack-based graph-traversal method: With

Exercises

- 5.92** Show how recursive depth-first search visits the nodes in the graph built for the edge sequence 0-2, 1-4, 2-5, 3-6, 0-4, 6-0, and 1-3 (see Exercise 3.70), by giving diagrams corresponding to Figures 5.33 (left) and 5.34 (right).
- 5.93** Show how stack-based depth-first search visits the nodes in the graph built for the edge sequence 0-2, 1-4, 2-5, 3-6, 0-4, 6-0, and 1-3, by giving diagrams corresponding to Figures 5.33 (left) and 5.34 (right).
- 5.94** Show how (queue-based) breadth-first search visits the nodes in the graph built for the edge sequence 0-2, 1-4, 2-5, 3-6, 0-4, 6-0, and 1-3, by giving diagrams corresponding to Figures 5.33 (right) and 5.35 (left).
- **5.95** Why is the running time in Property 5.10 quoted as $V + E$ and not simply E ?
- 5.96** Show how stack-based depth-first search visits the nodes in the example graph in the text (Figure 3.15) when using a forget-the-old-item policy, by giving diagrams corresponding to Figures 5.33 (left) and 5.35 (right).
- 5.97** Show how stack-based depth-first search visits the nodes in the example graph in the text (Figure 3.15) when using an ignore-the-new-item policy, by giving diagrams corresponding to Figures 5.33 (left) and 5.35 (right).
- ▷ **5.98** Implement a stack-based depth-first search for graphs that are represented with adjacency lists.
- **5.99** Implement a recursive depth-first search for graphs that are represented with adjacency lists.

5.9 Perspective

Recursion lies at the heart of early theoretical studies into the nature of computation. Recursive functions and programs play a central role in mathematical studies that attempt to separate problems that can be solved by a computer from problems that cannot be.

It is certainly impossible to do justice to topics as far-reaching as trees and recursion in so brief a discussion. Many of the best examples of recursive programs will be our focus throughout the book—divide-and-conquer algorithms and recursive data structures that have been applied successfully to solve a wide variety of problems. For many applications, there is no reason to go beyond a simple, direct recursive implementation; for others, we will consider the derivation of alternate nonrecursive and bottom-up implementations.

In this book, our interest lies in the practical aspects of recursive programs and data structures. Our goal is to exploit recursion to

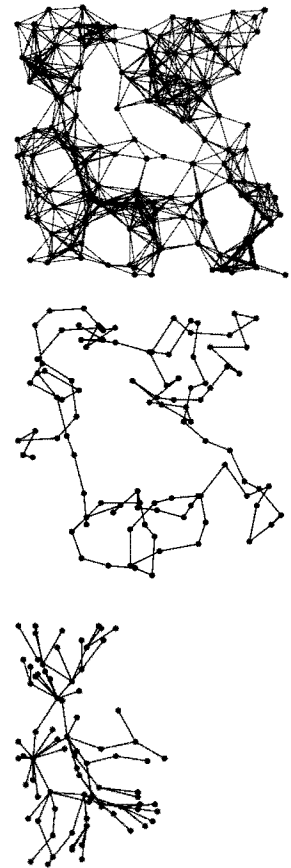


Figure 5.36
Graph-traversal trees

This diagram shows depth-first search (center) and breadth-first search (bottom), halfway through searching in a large graph (top). Depth-first search meanders from one node to the next, so most nodes are connected to just two others. By contrast, breadth-first search sweeps through the graph, visiting all the nodes connected to a given node before moving on, so several nodes are connected to many others.

produce elegant and efficient implementations. To meet that goal, we need to have particular respect for the dangers of simple programs that lead to an exponential number of function calls or impossibly deep nesting. Despite this pitfall, recursive programs and data structures are attractive because they often provide us with inductive arguments that can convince us that our programs are correct and efficient.

We use trees throughout the book, both to help us understand the dynamic properties of programs, and as dynamic data structures. Chapters 12 through 15 in particular are largely devoted to the manipulation of explicit tree structures. The properties described in this chapter provide us with the basic information that we need if we are to use explicit tree structures effectively.

Despite its central role in algorithm design, recursion is not a panacea. As we discovered in our study of tree- and graph-traversal algorithms, stack-based (inherently recursive) algorithms are not the only option when we have multiple computational tasks to manage. An effective algorithm-design technique for many problems is the use of generalized queue implementations other than stacks to give us the freedom to choose the next task according to some more subjective criteria than simply choosing the most recent. Data structures and algorithms that efficiently support such operations are a prime topic of Chapter 9, and we shall encounter many examples of their application when we consider graph algorithms in Part 7.

References for Part Two

There are numerous introductory textbooks on data structures. For example, the book by Standish covers linked structures, data abstraction, stacks and queues, memory allocation, and software engineering concepts at a more leisurely pace than here. Summit's book (and its source on the web) is an invaluable source of detailed information about C implementations, as is, of course, the Kernighan and Ritchie classic. The book by Plauger is a thorough explanation of C library functions.

The designers of PostScript perhaps did not anticipate that their language would be of interest to people learning basic algorithms and data structures. However, the language is not difficult to learn, and the reference manual is both thorough and accessible.

The technique for implementing ADTs with pointers to structures that are not specified was taught by Appel in the systems programming course at Princeton in the mid 1980s. It is described in full detail, with numerous examples, in the book by Hanson. The Hanson and Summit books are both outstanding references for programmers who want to write bugfree and portable code for large systems.

Knuth's books, particularly Volumes 1 and 3, remain the authoritative source on properties of elementary data structures. Baeza-Yates and Gonnet have more up-to-date information, backed by an extensive bibliography. Sedgewick and Flajolet cover mathematical properties of trees in detail.

Adobe Systems Incorporated, *PostScript Language Reference Manual*, second edition, Addison-Wesley, Reading, MA, 1990.

R. Baeza-Yates and G. H. Gonnet, *Handbook of Algorithms and Data Structures*, second edition, Addison-Wesley, Reading, MA, 1984.

D. R. Hanson, *C Interfaces and Implementations: Techniques for Creating Reusable Software*, Addison-Wesley, 1997.

B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, second edition, Prentice-Hall, Englewood Cliffs, NJ, 1988.

D. E. Knuth, *The Art of Computer Programming. Volume 1: Fundamental Algorithms*, second edition, Addison-Wesley, Reading, MA, 1973; *Volume 2: Seminumerical Algorithms*, second edition, Addison-Wesley, Reading, MA, 1981; *Volume 3: Sorting*

- and Searching*, second printing, Addison-Wesley, Reading, MA, 1975.
- P. J. Plauger, *The Standard C Library*, Prentice-Hall, Englewood Cliffs, NJ, 1992.
- R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, Addison-Wesley, Reading, MA, 1996.
- T. A. Standish, *Data Structures, Algorithms, and Software Principles in C*, Addison-Wesley, 1995.
- S. Summit, *C Programming FAQs*, Addison-Wesley, 1996.

P A R T
T H R E E

Sorting

Elementary Sorting Methods

FOR OUR FIRST excursion into the area of sorting algorithms, we shall study several elementary methods that are appropriate for small files, or for files that have a special structure. There are several reasons for studying these simple sorting algorithms in detail. First, they provide context in which we can learn terminology and basic mechanisms for sorting algorithms, and thus allow us to develop an adequate background for studying the more sophisticated algorithms. Second, these simple methods are actually more effective than the more powerful general-purpose methods in many applications of sorting. Third, several of the simple methods extend to better general-purpose methods or are useful in improving the efficiency of more sophisticated methods.

Our purpose in this chapter is not just to introduce the elementary methods, but also to develop a framework within which we can study sorting in later chapters. We shall look at a variety of situations that may be important in applying sorting algorithms, examine different kinds of input files, and look at other ways of comparing sorting methods and learning their properties.

We begin by looking at a simple driver program for testing sorting methods, which provides a context for us to consider the conventions that we shall follow. We also consider the basic properties of sorting methods that are important for us to know when we are evaluating the utility of algorithms for particular applications. Then, we look closely at implementations of three elementary methods: selection sort, insertion sort, and bubble sort. Following that, we examine

the performance characteristics of these algorithms in detail. Next, we look at shellsort, which is perhaps not properly characterized as elementary, but is easy to implement and is closely related to insertion sort. After a digression into the mathematical properties of shellsort, we delve into the subject of developing data type interfaces and implementations, along the lines that we have discussed in Chapters 3 and 4, for extending our algorithms to sort the kinds of data files that arise in practice. We then consider sorting methods that refer indirectly to the data and linked-list sorting. The chapter concludes with a discussion of a specialized method that is appropriate when the key values are known to be restricted to a small range.

In numerous sorting applications, a simple algorithm may be the method of choice. First, we often use a sorting program only once, or just a few times. Once we have “solved” a sort problem for a set of data, we may not need the sort program again in the application manipulating those data. If an elementary sort is no slower than some other part of processing the data—for example reading them in or printing them out—then there may be no point in looking for a faster way. If the number of items to be sorted is not too large (say, less than a few hundred elements), we might just choose to implement and run a simple method, rather than bothering with the interface to a system sort or with implementing and debugging a complicated method. Second, elementary methods are always suitable for small files (say, less than a few dozen elements)—sophisticated algorithms generally incur overhead that makes them slower than elementary ones for small files. This issue is not worth considering unless we wish to sort a huge number of small files, but applications with such a requirement are not unusual. Other types of files that are relatively easy to sort are ones that are already almost sorted (or already are sorted!) or ones that contain large numbers of duplicate keys. We shall see that several of the simple methods are particularly efficient when sorting such well-structured files.

As a rule, the elementary methods that we discuss here take time proportional to N^2 to sort N randomly arranged items. If N is small, this running time may be perfectly adequate. As just mentioned, the methods are likely to be even faster than more sophisticated methods for tiny files and in other special situations. But the methods that we discuss in this chapter are *not* suitable for large, randomly arranged

files, because the running time will become excessive even on the fastest computers. A notable exception is shellsort (see Section 6.6), which takes many fewer than N^2 steps for large N , and is arguably the sorting method of choice for midsize files and for a few other special applications.

6.1 Rules of the Game

Before considering specific algorithms, we will find it useful to discuss general terminology and basic assumptions for sorting algorithms. We shall be considering methods of sorting *files* of *items* containing *keys*. All these concepts are natural abstractions in modern programming environments. The keys, which are only part (often a small part) of the items, are used to control the sort. The objective of the sorting method is to rearrange the items such that their keys are ordered according to some well-defined ordering rule (usually numerical or alphabetical order). Specific characteristics of the keys and the items can vary widely across applications, but the abstract notion of putting keys and associated information into order is what characterizes the sorting problem.

If the file to be sorted will fit into memory, then the sorting method is called *internal*. Sorting files from tape or disk is called *external* sorting. The main difference between the two is that an internal sort can access any item easily whereas an external sort must access items sequentially, or at least in large blocks. We shall look at a few external sorts in Chapter 11, but most of the algorithms that we consider are internal sorts.

We shall consider both arrays and linked lists. The problem of sorting arrays and the problem of sorting linked lists are both of interest: during the development of our algorithms, we shall also encounter some basic tasks that are best suited for sequential allocation, and other tasks that are best suited for linked allocation. Some of the classical methods are sufficiently abstract that they can be implemented efficiently for either arrays or linked lists; others are particularly well suited to one or the other. Other types of access restrictions are also sometimes of interest.

We begin by focusing on array sorting. Program 6.1 illustrates many of the conventions that we shall use in our implementations. It

Program 6.1 Example of array sort with driver program

This program illustrates our conventions for implementing basic array sorts. The `main` function is a driver that initializes an array of integers (either with random values or from standard input), calls a `sort` function to sort that array, then prints out the ordered result.

The `sort` function, which is a version of insertion sort (see Section 6.3 for a detailed description, an example, and an improved implementation), assumes that the data type of the items being sorted is `Item`, and that the operations `less` (compare two keys), `exch` (exchange two items), and `compexch` (compare two items and exchange them if necessary to make the second not less than the first) are defined for `Item`. We implement `Item` for integers (as needed by `main`) with `typedef` and simple macros in this code. Use of other data types is the topic of Section 6.7, and does not affect `sort`.

```
#include <stdio.h>
#include <stdlib.h>
typedef int Item;
#define key(A) (A)
#define less(A, B) (key(A) < key(B))
#define exch(A, B) { Item t = A; A = B; B = t; }
#define compexch(A, B) if (less(B, A)) exch(A, B)
void sort(Item a[], int l, int r)
{
    int i, j;
    for (i = l+1; i <= r; i++)
        for (j = i; j > l; j--)
            compexch(a[j-1], a[j]);
}

main(int argc, char *argv[])
{
    int i, N = atoi(argv[1]), sw = atoi(argv[2]);
    int *a = malloc(N*sizeof(int));
    if (sw)
        for (i = 0; i < N; i++)
            a[i] = 1000*(1.0*rand()/RAND_MAX);
    else
        while (scanf("%d", &a[N]) == 1) N++;
    sort(a, 0, N-1);
    for (i = 0; i < N; i++) printf("%3d ", a[i]);
    printf("\n");
}
```

consists of a driver program that fills an array by reading integers from standard input or generating random ones (as dictated by an integer argument); then calls a sort function to put the integers in the array in order; then prints out the sorted result.

As we know from Chapters 3 and 4, there are numerous mechanisms available to us to arrange for our sort implementations to be useful for other types of data. We shall discuss the use of such mechanisms in detail in Section 6.7. The sort function in Program 6.1 uses a simple inline data type like the one discussed in Section 4.1, referring to the items being sorted only through its arguments and a few simple operations on the data. As usual, this approach allows us to use the same code to sort other types of items. For example, if the code for generating, storing, and printing random keys in the function `main` in Program 6.1 were changed to process floating-point numbers instead of integers, the only change that we would have to make outside of `main` is to change the `typedef` for `Item` from `int` to `float` (and we would not have to change `sort` at all). To provide such flexibility (while at the same time explicitly identifying those variables that hold items) our sort implementations will leave the data type of the items to be sorted unspecified as `Item`. For the moment, we can think of `Item` as `int` or `float`; in Section 6.7, we shall consider in detail data-type implementations that allow us to use our sort implementations for arbitrary items with floating-point numbers, strings, and other different types of keys, using mechanisms discussed in Chapters 3 and 4.

We can substitute for `sort` any of the array-sort implementations from this chapter, or from Chapters 7 through 10. They all assume that items of type `Item` are to be sorted, and they all take three arguments: the array, and the left and right bounds of the subarray to be sorted. They also all use `less` to compare keys in items and `exch` to exchange items (or the `compxch` combination). To differentiate sorting methods, we give our various sort routines different names. It is a simple matter to rename one of them, to change the driver, or to use function pointers to switch algorithms in a client program such as Program 6.1 without having to change any code in the sort implementation.

These conventions will allow us to examine natural and concise implementations of many array-sorting algorithms. In Sections 6.7 and 6.8, we shall consider a driver that illustrates how to use the implementations in more general contexts, and numerous data type

implementations. Although we are ever mindful of such packaging considerations, our focus will be on algorithmic issues, to which we now turn.

The example sort function in Program 6.1 is a variant of *insertion sort*, which we shall consider in detail in Section 6.3. Because it uses only compare-exchange operations, it is an example of a *nonadaptive* sort: The sequence of operations that it performs is independent of the order of the data. By contrast, an *adaptive* sort is one that performs different sequences of operations, depending on the outcomes of comparisons (less operations). Nonadaptive sorts are interesting because they are well suited for hardware implementation (see Chapter 11), but most of the general-purpose sorts that we consider are adaptive.

As usual, the primary performance parameter of interest is the running time of our sorting algorithms. The selection-sort, insertion-sort, and bubble-sort methods that we discuss in Sections 6.2 through 6.4 all require time proportional to N^2 to sort N items, as discussed in Section 6.5. The more advanced methods that we discuss in Chapters 7 through 10 can sort N items in time proportional to $N \log N$, but they are not always as good as the methods considered here for small N and in certain other special situations. In Section 6.6, we shall look at a more advanced method (shellsort) that can run in time proportional to $N^{3/2}$ or less, and, in Section 6.10, we shall see a specialized method (key-indexed sorting) that runs in time proportional to N for certain types of keys.

The analytic results described in the previous paragraph all follow from enumerating the basic operations (comparisons and exchanges) that the algorithms perform. As discussed in Section 2.2, we also must consider the costs of the operations, and we generally find it worthwhile to focus on the most frequently executed operations (the inner loop of the algorithm). Our goal is to develop efficient and reasonable implementations of efficient algorithms. In pursuit of this goal, we will not just avoid gratuitous additions to inner loops, but also look for ways to remove instructions from inner loops when possible. Generally, the best way to reduce costs in an application is to switch to a more efficient algorithm; the second best way is to tighten the inner loop. We shall consider both options in detail for sorting algorithms.

The amount of extra memory used by a sorting algorithm is the second important factor that we shall consider. Basically, the methods divide into three types: those that sort in place and use no extra memory except perhaps for a small stack or table; those that use a linked-list representation or otherwise refer to data through pointers or array indices, and so need extra memory for N pointers or indices; and those that need enough extra memory to hold another copy of the array to be sorted.

We frequently use sorting methods for items with multiple keys—we may even need to sort one set of items using different keys at different times. In such cases, it may be important for us to be aware whether or not the sorting method that we use has the following property:

Definition 6.1 *A sorting method is said to be stable if it preserves the relative order of items with duplicated keys in the file.*

For example, if an alphabetized list of students and their year of graduation is sorted by year, a stable method produces a list in which people in the same class are still in alphabetical order, but a nonstable method is likely to produce a list with no vestige of the original alphabetic order. Figure 6.1 shows an example. Often, people who are unfamiliar with stability are surprised by the way an unstable algorithm seems to scramble the data when they first encounter the situation.

Several (but not all) of the simple sorting methods that we consider in this chapter are stable. On the other hand, many (but not all) of the sophisticated algorithms that we consider in the next several chapters are not. If stability is vital, we can force it by appending a small index to each key before sorting or by lengthening the sort key in some other way. Doing this extra work is tantamount to using both keys for the sort in Figure 6.1; using a stable algorithm would be preferable. It is easy to take stability for granted; actually, few of the sophisticated methods that we see in later chapters achieve stability without using significant extra time or space.

As we have mentioned, sorting programs normally access items in one of two ways: either keys are accessed for comparison, or entire items are accessed to be moved. If the items to be sorted are large, it is wise to avoid shuffling them around by doing an *indirect sort*: we rearrange not the items themselves, but rather an array of pointers (or indices) such that the first pointer points to the smallest item, the

Adams	1
Black	2
Brown	4
Jackson	2
Jones	4
Smith	1
Thompson	4
Washington	2
White	3
Wilson	3

Adams	1
Smith	1
Washington	2
Jackson	2
Black	2
White	3
Wilson	3
Thompson	4
Brown	4
Jones	4

Adams	1
Smith	1
Black	2
Jackson	2
Washington	2
White	3
Wilson	3
Brown	4
Jones	4
Thompson	4

Figure 6.1
Stable-sort example

A sort of these records might be appropriate on either key. Suppose that they are sorted initially by the first key (top). A nonstable sort on the second key does not preserve the order in records with duplicate keys (center), but a stable sort does preserve the order (bottom).

second pointer points to the next smallest item, and so forth. We can keep keys either with the items (if the keys are large) or with the pointers (if the keys are small). We could rearrange the items after the sort, but that is often unnecessary, because we do have the capability to refer to them in sorted order (indirectly). We shall consider indirect sorting in Section 6.8.

Exercises

- ▷ 6.1 A child's sorting toy has i cards that fit on a peg in position i for i from 1 to 5. Write down the method that you use to put the cards on the pegs, assuming that you cannot tell from the card whether it fits on a peg (you have to try fitting it on).
- 6.2 A card trick requires that you put a deck of cards in order by suit (in the order spades, hearts, clubs, diamonds) and by rank within each suit. Ask a few friends to do this task (shuffling in between!) and write down the method(s) that they use.
- 6.3 Explain how you would sort a deck of cards with the restriction that the cards must be laid out face down in a row, and the only allowed operations are to check the values of two cards and (optionally) to exchange them.
- 6.4 Explain how you would sort a deck of cards with the restriction that the cards must be kept stacked in the deck, and the only allowed operations are to look at the value of the top two cards, to exchange the top two cards, and to move the top card to the bottom of the deck.
- 6.5 Give all sequences of three compare-exchange operations that will sort three elements (see Program 6.1).
- 6.6 Give a sequence of five compare-exchange operations that will sort four elements.
- 6.7 Write a client program that checks whether the sort routine being used is stable.
- 6.8 Checking that the array is sorted after `sort` provides no guarantee that the sort works. Why not?
- 6.9 Write a performance driver client program that runs `sort` multiple times on files of various sizes, measures the time taken for each run, and prints out or plots the average running times.
- 6.10 Write an exercise driver client program that runs `sort` on difficult or pathological cases that might turn up in practical applications. Examples include files that are already in order, files in reverse order, files where all keys are the same, files consisting of only two distinct values, and files of size 0 or 1.

6.2 Selection Sort

One of the simplest sorting algorithms works as follows. First, find the smallest element in the array, and exchange it with the element in the first position. Then, find the second smallest element and exchange it with the element in the second position. Continue in this way until the entire array is sorted. This method is called *selection sort* because it works by repeatedly selecting the smallest remaining element. Figure 6.2 shows the method in operation on a sample file.

Program 6.2 is an implementation of selection sort that adheres to our conventions. The inner loop is just a comparison to test a current element against the smallest element found so far (plus the code necessary to increment the index of the current element and to check that it does not exceed the array bounds); it could hardly be simpler. The work of moving the items around falls outside the inner loop: each exchange puts an element into its final position, so the number of exchanges is $N - 1$ (no exchange is needed for the final element). Thus the running time is dominated by the number of comparisons. In Section 6.5, we show this number to be proportional to N^2 , and examine more closely how to predict the total running time and how to compare selection sort with other elementary sorts.

A disadvantage of selection sort is that its running time depends only slightly on the amount of order already in the file. The process of finding the minimum element on one pass through the file does not seem to give much information about where the minimum might be on the next pass through the file. For example, the user of the sort might be surprised to realize that it takes about as long to run selection sort for a file that is already in order, or for a file with all keys equal, as it does for a randomly ordered file! As we shall see, other methods are better able to take advantage of order in the input file.

Despite its simplicity and evident brute-force approach, selection sort outperforms more sophisticated methods in one important application: it is the method of choice for sorting files with huge items and small keys. For such applications, the cost of moving the data dominates the cost of making comparisons, and no algorithm can sort a file with substantially less data movement than selection sort (see Property 6.5 in Section 6.5).

```

A S O R T I N G E X A M P L E
A S O R T I N G E X A M P L E
A A O R T I N G E X S M P L E
A A E R T I N G O X S M P L E
A A E E T I N G O X S M P L R
A A E E G I N T O X S M P L R
A A E E G I N T O X S M P L R
A A E E G I L T O X S M P N R
A A E E G I L M O X S T P N R
A A E E G I L M N X S T P O R
A A E E G I L M N O S T P X R
A A E E G I L M N O P T S X R
A A E E G I L M N O P R S X T
A A E E G I L M N O P R S X T
A A E E G I L M N O P R S T X
A A E E G I L M N O P R S T X

```

Figure 6.2
Selection sort example

The first pass has no effect in this example, because there is no element in the array smaller than the A at the left. On the second pass, the other A is the smallest remaining element, so it is exchanged with the S in the second position. Then, the E near the middle is exchanged with the O in the third position on the third pass; then, the other E is exchanged with the R in the fourth position on the fourth pass; and so forth.

Program 6.2 Selection sort

For each i from 1 to $r-1$, exchange $a[i]$ with the minimum element in $a[i], \dots, a[r]$. As the index i travels from left to right, the elements to its left are in their final position in the array (and will not be touched again), so the array is fully sorted when i reaches the right end.

```
void selection(Item a[], int l, int r)
{
    int i, j;
    for (i = l; i < r; i++)
    {
        int min = i;
        for (j = i+1; j <= r; j++)
            if (less(a[j], a[min])) min = j;
        exch(a[i], a[min]);
    }
}
```

Exercises

- ▷ 6.11 Show, in the style of Figure 6.2, how selection sort sorts the sample file EASYQUESTION.
- 6.12 What is the maximum number of exchanges involving any particular element during selection sort? What is the average number of exchanges involving an element?
- 6.13 Give an example of a file of N elements that maximizes the number of times the test `less(a[j], a[min])` fails (and, therefore, `min` gets updated) during the operation of selection sort.
- 6.14 Is selection sort stable?

6.3 Insertion Sort

The method that people often use to sort bridge hands is to consider the elements one at a time, inserting each into its proper place among those already considered (keeping them sorted). In a computer implementation, we need to make space for the element being inserted by moving larger elements one position to the right, and then inserting the element into the vacated position. The sort function in Program 6.1 is an implementation of this method, which is called *insertion sort*.

As in selection sort, the elements to the left of the current index are in sorted order during the sort, but they are not in their final

position, as they may have to be moved to make room for smaller elements encountered later. The array is, however, fully sorted when the index reaches the right end. Figure 6.3 shows the method in operation on a sample file.

The implementation of insertion sort in Program 6.1 is straightforward, but inefficient. We shall now consider three ways to improve it, to illustrate a recurrent theme throughout many of our implementations: We want code to be succinct, clear, and efficient, but these goals sometimes conflict, so we must often strike a balance. We do so by developing a natural implementation, then seeking to improve it by a sequence of transformations, checking the effectiveness (and correctness) of each transformation.

First, we can stop doing `compexch` operations when we encounter a key that is not larger than the key in the item being inserted, because the subarray to the left is sorted. Specifically, we can break out of the inner `for` loop in `sort` in Program 6.1 when the condition `less(a[j-1], a[j])` is true. This modification changes the implementation into an adaptive sort, and speeds up the program by about a factor of 2 for randomly ordered keys (see Property 6.2).

With the improvement described in the previous paragraph, we have two conditions that terminate the inner loop—we could recode it as a `while` loop to reflect that fact explicitly. A more subtle improvement of the implementation follows from noting that the test `j > 1` is usually extraneous: indeed, it succeeds *only* when the element inserted is the smallest seen so far and reaches the beginning of the array. A commonly used alternative is to keep the keys to be sorted in `a[1]` to `a[N]`, and to put a *sentinel key* in `a[0]`, making it at least as small as the smallest key in the array. Then, the test whether a smaller key has been encountered simultaneously tests both conditions of interest, making the inner loop smaller and the program faster.

Sentinels are sometimes inconvenient to use: perhaps the smallest possible key is not easily defined, or perhaps the calling routine has no room to include an extra key. Program 6.3 illustrates one way around these two problems for insertion sort: We make an explicit first pass over the array that puts the item with the smallest key in the first position. Then, we sort the rest of the array, with that first and smallest item now serving as sentinel. We generally shall avoid sentinels in our code, because it is often easier to understand code with

```

A S O R T I N G E X A M P L E
A (S) O R T I N G E X A M P L E
A (O) S R T I N G E X A M P L E
A O (R) S T I N G E X A M P L E
A O R S (T) I N G E X A M P L E
A (I) O R S T I N G E X A M P L E
A I (N) O R S T I N G E X A M P L E
A (G) I N O R S T I N G E X A M P L E
A (E) G I N O R S T I N G E X A M P L E
A E G I N O R S T (X) A M P L E
A (A) E G I N O R S T I N G E X A M P L E
A A E G I (M) N O R S T I N G E X A M P L E
A A E G I M N O (P) R S T I N G E X A M P L E
A A E G I (L) M N O P R S T I N G E X A M P L E
A A E (E) G I L M N O P R S T I N G E X A M P L E
A A E E G I L M N O P R S T I N G E X A M P L E

```

Figure 6.3
Insertion sort example

During the first pass of insertion sort, the S in the second position is larger than the A, so it does not have to be moved. On the second pass, when the O in the third position is encountered, it is exchanged with the S to put A O S in sorted order, and so forth. Unshaded elements that are not circled are those that were moved one position to the right.

Program 6.3 Insertion sort

This code is an improvement over the implementation of `sort` in Program 6.1 because (i) it first puts the smallest element in the array into the first position, so that that element can serve as a sentinel; (ii) it does a single assignment, rather than an exchange, in the inner loop; and (iii) it terminates the inner loop when the element being inserted is in position. For each i , it sorts the elements $a[1], \dots, a[i]$ by moving one position to the right elements in the sorted list $a[1], \dots, a[i-1]$ that are larger than $a[i]$, then putting $a[i]$ into its proper position.

```
void insertion(Item a[], int l, int r)
{ int i;
  for (i = r; i > l; i--) compexch(a[i-1], a[i]);
  for (i = l+2; i <= r; i++)
  { int j = i; Item v = a[i];
    while (less(v, a[j-1]))
      { a[j] = a[j-1]; j--; }
    a[j] = v;
  }
}
```

explicit tests, but we shall note situations where sentinels might be useful in making programs both simpler and more efficient.

The third improvement that we shall consider also involves removing extraneous instructions from the inner loop. It follows from noting that successive exchanges involving the same element are inefficient. If there are two or more exchanges, we have

```
t = a[j]; a[j] = a[j-1]; a[j-1] = t;
```

followed by

```
t = a[j-1]; a[j-1] = a[j-2]; a[j-2] = t;
```

and so forth. The value of t does not change between these two sequences, and we waste time storing it, then reloading it for the next exchange. Program 6.3 moves larger elements one position to the right instead of using exchanges, and thus avoids wasting time in this way.

Program 6.3 is an implementation of insertion sort that is more efficient than the one given in Program 6.1 (in Section 6.5, we shall see that it is nearly twice as fast). In this book, we are interested *both* in elegant and efficient algorithms *and* in elegant and efficient imple-

mentations of them. In this case, the underlying algorithms do differ slightly—we should properly refer to the sort function in Program 6.1 as a *nonadaptive insertion sort*. A good understanding of the properties of an algorithm is the best guide to developing an implementation that can be used effectively in an application.

Unlike that of selection sort, the running time of insertion sort primarily depends on the initial order of the keys in the input. For example, if the file is large and the keys are already in order (or even are nearly in order), then insertion sort is quick and selection sort is slow. We compare the algorithms in more detail in Section 6.5.

Exercises

- ▷ 6.15 Show, in the style of Figure 6.3, how insertion sort sorts the sample file EASYQUESTION.
- 6.16 Give an implementation of insertion sort with the inner loop coded as a `while` loop that terminates on one of two conditions, as described in the text.
- 6.17 For each of the conditions in the `while` loop in Exercise 6.16, describe a file of N elements where that condition is always false when the loop terminates.
- 6.18 Is insertion sort stable?
- 6.19 Give a nonadaptive implementation of selection sort based on finding the minimum element with code like the first `for` loop in Program 6.3.

6.4 Bubble Sort

The first sort that many people learn, because it is so simple, is *bubble sort*: Keep passing through the file, exchanging adjacent elements that are out of order, continuing until the file is sorted. Bubble sort's prime virtue is that it is easy to implement, but whether it is actually easier to implement than insertion or selection sort is arguable. Bubble sort generally will be slower than the other two methods, but we consider it briefly for the sake of completeness.

Suppose that we always move from right to left through the file. Whenever the minimum element is encountered during the first pass, we exchange it with each of the elements to its left, eventually putting it into position at the left end of the array. Then on the second pass, the second smallest element will be put into position, and so forth. Thus, N passes suffice, and bubble sort operates as a type of selection

```

A S O R T I N G E X A M P L E
A (A) S O R T I N G E X (E) M P L
A A (E) S O R T I N G E X L M P
A A E (E) S O R T I N G L X (M) P
A A E E (G) S O R T I N L M X P
A A E E G (I) S O R T L N M P X
A A E E G I (L) S O R T M N P X
A A E E G I L (M) S O R T N P X
A A E E G I L M (N) S O R T P X
A A E E G I L M N (O) S P R T X
A A E E G I L M N O (P) S R T X
A A E E G I L M N O P (R) S T X
A A E E G I L M N O P R (S) T X
A A E E G I L M N O P R S (T) X
A A E E G I L M N O P R S T (X)

```

Figure 6.4
Bubble sort example

Small keys percolate over to the left in bubble sort. As the sort moves from right to left, each key is exchanged with the one on its left until a smaller one is encountered. On the first pass, the E is exchanged with the L, the P, and the M before stopping at the A on the right; then the A moves to the beginning of the file, stopping at the other A, which is already in position. The i th smallest key reaches its final position after the i th pass, just as in selection sort, but other keys are moved closer to their final position, as well.

Program 6.4 Bubble sort

For each i from 1 to $r-1$, the inner (j) loop puts the minimum element among the elements in $a[i], \dots, a[r]$ into $a[i]$ by passing from right to left through the elements, compare-exchanging successive elements. The smallest one moves on all such comparisons, so it “bubbles” to the beginning. As in selection sort, as the index i travels from left to right through the file, the elements to its left are in their final position in the array.

```

void bubble(Item a[], int l, int r)
{ int i, j;
  for (i = l; i < r; i++)
    for (j = r; j > i; j--)
      compexch(a[j-1], a[j]);
}

```

sort, although it does more work to get each element into position. Program 6.4 is an implementation, and Figure 6.4 shows an example of the algorithm in operation.

We can speed up Program 6.4 by carefully implementing the inner loop, in much the same way as we did in Section 6.3 for insertion sort (see Exercise 6.25). Indeed, comparing the code, Program 6.4 appears to be virtually identical to the nonadaptive insertion sort in Program 6.1. The difference between the two is that the inner `for` loop moves through the left (sorted) part of the array for insertion sort and through the right (not necessarily sorted) part of the array for bubble sort.

Program 6.4 uses only `compexch` instructions and is therefore nonadaptive, but we can improve it to run more efficiently when the file is nearly in order by testing whether no exchanges at all are performed on one of the passes (and therefore the file is in sorted order, so we can break out of the outer loop). Adding this improvement will make bubble sort faster on some types of files, but it is generally not as effective as is changing insertion sort to break out of the inner loop, as discussed in detail in Section 6.5.

Exercises

- ▷ 6.20 Show, in the style of Figure 6.4, how bubble sort sorts the sample file EASYQUESTION.

- 6.21** Give an example of a file for which the number of exchanges done by bubble sort is maximized.
- **6.22** Is bubble sort stable?
- 6.23** Explain how bubble sort is preferable to the nonadaptive version of selection sort described in Exercise 6.19.
- **6.24** Do experiments to determine how many passes are saved, for random files of N elements, when you add to bubble sort a test to terminate when the file is sorted.
- 6.25** Develop an efficient implementation of bubble sort, with as few instructions as possible in the inner loop. Make sure that your “improvements” do not slow down the program!

6.5 Performance Characteristics of Elementary Sorts

Selection sort, insertion sort, and bubble sort are all quadratic-time algorithms both in the worst and in the average case, and none requires extra memory. Their running times thus differ by only a constant factor, but they operate quite differently, as illustrated in Figures 6.5 through 6.7.

Generally, the running time of a sorting algorithm is proportional to the number of comparisons that the algorithm uses, to the number of times that items are moved or exchanged, or to both. For random input, comparing the methods involves studying constant-factor differences in the numbers of comparisons and exchanges and constant-factor differences in the lengths of the inner loops. For input with special characteristics, the running times of the methods may differ by more than a constant factor. In this section, we look closely at the analytic results in support of this conclusion.

Property 6.1 *Selection sort uses about $N^2/2$ comparisons and N exchanges.*

We can verify this property easily by examining the sample run in Figure 6.2, which is an N -by- N table in which unshaded letters correspond to comparisons. About one-half of the elements in the table are unshaded—those above the diagonal. The $N - 1$ (not the final one) elements on the diagonal each correspond to an exchange. More precisely, examination of the code reveals that, for each i from 1 to $N - 1$, there is one exchange and $N - i$ comparisons, so there is a total

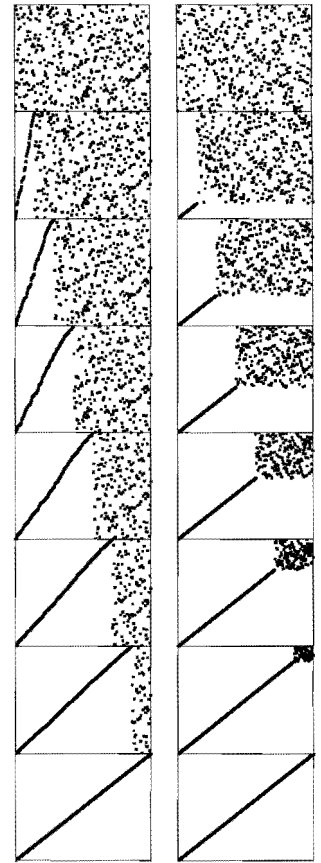


Figure 6.5
Dynamic characteristics of insertion and selection sorts

These snapshots of insertion sort (left) and selection sort (right) in action on a random permutation illustrate how each method progresses through the sort. We represent an array being sorted by plotting i vs. $a[i]$ for each i . Before the sort, the plot is uniformly random; after the sort, it is a diagonal line from bottom left to top right. Insertion sort never looks ahead of its current position in the array; selection sort never looks back.

of $N - 1$ exchanges and $(N - 1) + (N - 2) + \cdots + 2 + 1 = N(N - 1)/2$ comparisons. These observations hold no matter what the input data are; the only part of selection sort that does depend on the input is the number of times that `min` is updated. In the worst case, this quantity could also be quadratic; in the average case, however, it is just $O(N \log N)$ (see reference section), so we can expect the running time of selection sort to be insensitive to the input. ■

Property 6.2 *Insertion sort uses about $N^2/4$ comparisons and $N^2/4$ half-exchanges (moves) on the average, and twice that many at worst.*

As implemented in Program 6.3, the number of comparisons and of moves is the same. Just as for Property 6.1, this quantity is easy to visualize in the N -by- N diagram in Figure 6.3 that gives the details of the operation of the algorithm. Here, the elements below the diagonal are counted—all of them, in the worst case. For random input, we expect each element to go about halfway back, on the average, so one-half of the elements below the diagonal should be counted. ■

Property 6.3 *Bubble sort uses about $N^2/2$ comparisons and $N^2/2$ exchanges on the average and in the worst case.*

The i th bubble sort pass requires $N - i$ compare-exchange operations, so the proof goes as for selection sort. When the algorithm is modified to terminate when it discovers that the file is sorted, the running time depends on the input. Just one pass is required if the file is already in order, but the i th pass requires $N - i$ comparisons *and* exchanges if the file is in reverse order. The average-case performance is not significantly better than the worst case, as stated, although the analysis that demonstrates this fact is complicated (see reference section). ■

Although the concept of a partially sorted file is necessarily rather imprecise, insertion sort and bubble sort work well for certain types of nonrandom files that often arise in practice. General-purpose sorts are commonly misused for such applications. For example, consider the operation of insertion sort on a file that is already sorted. Each element is immediately determined to be in its proper place in the file, and the total running time is linear. The same is true for bubble sort, but selection sort is still quadratic.

Definition 6.2 *An inversion is a pair of keys that are out of order in the file.*

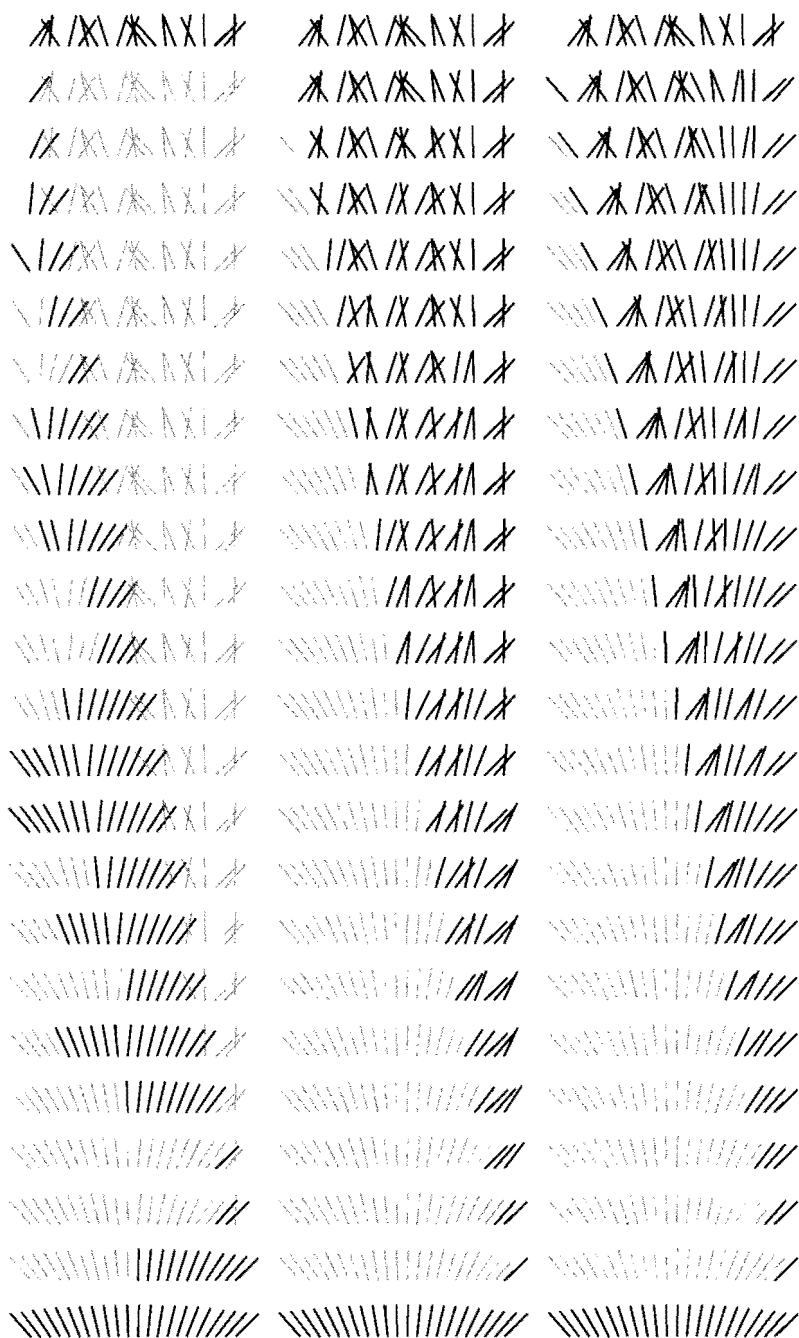


Figure 6.6
Comparisons and exchanges
in elementary sorts

This diagram highlights the differences in the way that insertion sort, selection sort, and bubble sort bring a file into order. The file to be sorted is represented by lines that are to be sorted according to their angles. Black lines correspond to the items accessed during each pass of the sort; gray lines correspond to items not touched. For insertion sort (left), the element to be inserted goes about halfway back through the sorted part of the file on each pass. Selection sort (center) and bubble sort (right) both go through the entire unsorted part of the array to find the next smallest element there for each pass; the difference between the methods is that bubble sort exchanges any adjacent out-of-order elements that it encounters, whereas selection sort just exchanges the minimum into position. The effect of this difference is that the unsorted part of the array becomes more nearly sorted as bubble sort progresses.

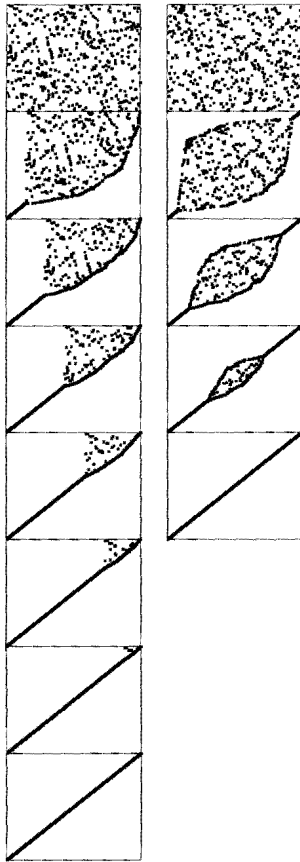


Figure 6.7
Dynamic characteristics of
two bubble sorts

Standard bubble sort (left) operates in a manner similar to selection sort in that each pass brings one element into position, but it also brings some order into other parts of the array, in an asymmetric manner. Changing the scan through the array to alternate between beginning to end and end to beginning gives a version of bubble sort called shaker sort (right), which finishes more quickly (see Exercise 6.30).

To count the number of inversions in a file, we can add up, for each element, the number of elements to its left that are greater (we refer to this quantity as the number of inversions corresponding to the element). But this count is precisely the distance that the elements have to move when inserted into the file during insertion sort. A file that has some order will have fewer inversions than will one that is arbitrarily scrambled.

In one type of partially sorted file, each item is close to its final position in the file. For example, some people sort their hand in a card game by first organizing the cards by suit, to put their cards close to their final position, then considering the cards one by one. We shall be considering a number of sorting methods that work in much the same way—they bring elements close to final positions in early stages to produce a partially sorted file with every element not far from where it ultimately must go. Insertion sort and bubble sort (but not selection sort) are efficient methods for sorting such files.

Property 6.4 *Insertion sort and bubble sort use a linear number of comparisons and exchanges for files with at most a constant number of inversions corresponding to each element.*

As just mentioned, the running time of insertion sort is directly proportional to the number of inversions in the file. For bubble sort (here, we are referring to Program 6.4, modified to terminate when the file is sorted), the proof is more subtle (see Exercise 6.29). Each bubble sort pass reduces the number of smaller elements to the right of any element by precisely 1 (unless the number was already 0), so bubble sort uses at most a constant number of passes for the types of files under consideration, and therefore does at most a linear number of comparisons and exchanges. ■

In another type of partially sorted file, we perhaps have appended a few elements to a sorted file or have edited a few elements in a sorted file to change their keys. This kind of file is prevalent in sorting applications. Insertion sort is an efficient method for such files; bubble sort and selection sort are not.

Property 6.5 *Insertion sort uses a linear number of comparisons and exchanges for files with at most a constant number of elements having more than a constant number of corresponding inversions.*

Table 6.1 Empirical study of elementary sorting algorithms

Insertion sort and selection sort are about twice as fast as bubble sort for small files, but running times grow quadratically (when the file size grows by a factor of 2, the running time grows by a factor of 4). None of the methods are useful for large randomly ordered files—for example, the numbers corresponding to those in this table are less than 2 for the shellsort algorithm in Section 6.6. When comparisons are expensive—for example, when the keys are strings—then insertion sort is much faster than the other two because it uses many fewer comparisons. Not included here is the case where exchanges are expensive; then selection sort is best.

<i>N</i>	32-bit integer keys					string keys		
	S	I*	I	B	B*	S	I	B
1000	5	7	4	11	8	13	8	19
2000	21	29	15	45	34	56	31	78
4000	85	119	62	182	138	228	126	321

Key:

- S Selection sort (Program 6.2)
- I* Insertion sort, exchange-based (Program 6.1)
- I Insertion sort (Program 6.3)
- B Bubble sort (Program 6.4)
- B* Shaker sort (Exercise 6.30)

The running time of insertion sort depends on the total number of inversions in the file, and does not depend on the way in which the inversions are distributed. ■

To draw conclusions about running time from Properties 6.1 through 6.5, we need to analyze the relative cost of comparisons and exchanges, a factor that in turn depends on the size of the items and keys (see Table 6.1). For example, if the items are one-word keys, then an exchange (four array accesses) should be about twice as expensive as a comparison. In such a situation, the running times of selection and insertion sort are roughly comparable, but bubble sort is slower. But if the items are large in comparison to the keys, then selection sort will be best.

Property 6.6 *Selection sort runs in linear time for files with large items and small keys.*

Let M be the ratio of the size of the item to the size of the key. Then we can assume the cost of a comparison to be 1 time unit and the cost of an exchange to be M time units. Selection sort takes about $N^2/2$ time units for comparisons and about NM time units for exchanges. If M is larger than a constant multiple of N , then the NM term dominates the N^2 term, so the running time is proportional to NM , which is proportional to the amount of time that would be required to move all the data. ■

For example, if we have to sort 1000 items that consist of 1-word keys and 1000 words of data each, and we actually have to rearrange the items, then we cannot do better than selection sort, since the running time will be dominated by the cost of moving all 1 million words of data. In Section 6.8, we shall see alternatives to rearranging the data.

Exercises

- ▷ **6.26** Which of the three elementary methods (selection sort, insertion sort, or bubble sort) runs fastest for a file with all keys identical?
- 6.27** Which of the three elementary methods runs fastest for a file in reverse order?
- 6.28** Give an example of a file of 10 elements (use the keys A through J) for which bubble sort uses fewer comparisons than insertion sort, or prove that no such file exists.
- **6.29** Show that each bubble sort pass reduces by precisely 1 the number of elements to the left of each element that are greater (unless that number was already 0).
- 6.30** Implement a version of bubble sort that alternates left-to-right and right-to-left passes through the data. This (faster but more complicated) algorithm is called *shaker sort* (see Figure 6.7).
- **6.31** Show that Property 6.5 does not hold for shaker sort (see Exercise 6.30).
- **6.32** Implement selection sort in PostScript (see Section 4.3), and use your implementation to draw figures like Figures 6.5 through 6.7. You may try a recursive implementation, or read the manual to learn about loops and arrays in PostScript.

6.6 Shellsort

Insertion sort is slow because the only exchanges it does involve adjacent items, so items can move through the array only one place at a time. For example, if the item with the smallest key happens to be at the end of the array, N steps are needed to get it where it belongs. *Shellsort* is a simple extension of insertion sort that gains speed by allowing exchanges of elements that are far apart.

The idea is to rearrange the file to give it the property that taking every h th element (starting anywhere) yields a sorted file. Such a file is said to be *h -sorted*. Put another way, an h -sorted file is h independent sorted files, interleaved together. By h -sorting for some large values of h , we can move elements in the array long distances and thus make it easier to h -sort for smaller values of h . Using such a procedure for any sequence of values of h that ends in 1 will produce a sorted file: that is the essence of shellsort.

One way to implement shellsort would be, for each h , to use insertion sort independently on each of the h subfiles. Despite the apparent simplicity of this process, we can use an even simpler approach, precisely because the subfiles are independent. When h -sorting the file, we simply insert it among the previous elements in its h -subfile by moving larger elements to the right (see Figure 6.8). We accomplish this task by using the insertion-sort code, but modified to increment or decrement by h instead of 1 when moving through the file. This observation reduces the shellsort implementation to nothing more than an insertion-sort-like pass through the file for each increment, as in Program 6.5. The operation of this program is illustrated in Figure 6.9.

How do we decide what increment sequence to use? In general, this question is a difficult one to answer. Properties of many different increment sequences have been studied in the literature, and some have been found that work well in practice, but no provably best sequence has been found. In practice, we generally use sequences that decrease roughly geometrically, so the number of increments is logarithmic in the size of the file. For example, if each increment is about one-half of the previous, then we need only about 20 increments to sort a file of 1 million elements; if the ratio is about one-quarter, then 10 increments will suffice. Using as few increments as possible is an important consideration that is easy to respect—we also need to

```

A S O R T I N G E X A M P L E
A S O R E I N G T X A M P L E
A S O R E I N G P X A M T L E

A I O R E S N G P X A M T L E
A I O R E S N G P X A M T L E
A I O R E L N G P S A M T X E

A I N R E L O G P S A M T X E
A I A R E L N G P S O M T X E
A I A R E L E G P S N M T X O

A I A G E L E R P S N M T X O
A I A G E L E M P S N R T X O

A S O R T I N G E X A M P L E
A I O R T S N G E X A M P L E
A I N R T S O G E X A M P L E
A I N G T S O R E X A M P L E
A I N G E S O R T X A M P L E
A I N G E S O R T X A M P L E
A I A G E S N R T X O M P L E
A I A G E S N M T X O R P L E
A I A G E S N M P X O R T L E
A I A G E L N M P S O R T X E
A I A G E L E M P S N R T X O

```

Figure 6.8
Interleaving 4-sorts

The top part of this diagram shows the process of 4-sorting a file of 15 elements by first insertion sorting the subfile at positions 0, 4, 8, 12, then insertion sorting the subfile at positions 1, 5, 9, 13, then insertion sorting the subfile at positions 2, 6, 10, 14, then insertion sorting the subfile at positions 3, 7, 11. But the four subfiles are independent, so we can achieve the same result by inserting each element into position into its subfile, going back four at a time (bottom). Taking the first row in each section of the top diagram, then the second row in each section, and so forth, gives the bottom diagram.

```

A S O R T I N G E X A M P L E
A S O R T I N G E X A M P L E
A E O R T I N G E X A M P L S

```

```

A E O R T I N G E X A M P L S
A E O R T I N G E X A M P L S
A E N R T I O G E X A M P L S
A E N G T I O R E X A M P L S
A E N G E I O R T X A M P L S
A E A G E I N R T X O M P L S
A E A G E I N M T X O R P L S
A E A G E I N M P X O R T L S
A E A G E I N M P L O R T X S
A E A G E I N M P L O R T X S

```

```

A E A G E I N M P L O R T X S
A A E E G I N M P L O R T X S
A A E E G I N M P L O R T X S
A A E E G I N M P L O R T X S
A A E E G I N M P L O R T X S
A A E E G I M N P L O R T X S
A A E E G I M N P L O R T X S
A A E E G I L M N P O R T X S
A A E E G I L M N O P R T X S
A A E E G I L M N O P R T X S
A A E E G I L M N O P R T X S
A A E E G I L M N O P R T X S
A A E E G I L M N O P R S T X
A A E E G I L M N O P R S T X

```

Figure 6.9
Shellsort example

Sorting a file by 13-sorting (top), then 4-sorting (center), then 1-sorting (bottom) does not involve many comparisons (as indicated by the unshaded elements). The final pass is just insertion sort, but no element has to move far because of the order in the file due to the first two passes.

Program 6.5 Shellsort

If we do not use sentinels and then replace every occurrence of “1” by “h” in insertion sort, the resulting program *h*-sorts the file. Adding an outer loop to change the increments leads to this compact shellsort implementation, which uses the increment sequence 1 4 13 40 121 364 1093 3280 9841

```

void shellsort(Item a[], int l, int r)
{ int i, j, h;
  for (h = 1; h <= (r-l)/9; h = 3*h+1) ;
  for ( ; h > 0; h /= 3)
    for (i = l+h; i <= r; i++)
      { int j = i; Item v = a[i];
        while (j >= l+h && less(v, a[j-h]))
          { a[j] = a[j-h]; j -= h; }
        a[j] = v;
      }
}

```

consider arithmetical interactions among the increments such as the size of their common divisors and other properties.

The practical effect of finding a good increment sequence is limited to perhaps a 25% speedup, but the problem presents an intriguing puzzle that provides a good example of the inherent complexity in an apparently simple algorithm.

The increment sequence 1 4 13 40 121 364 1093 3280 9841 ... that is used in Program 6.5, with a ratio between increments of about one-third, was recommended by Knuth in 1969 (*see reference section*). It is easy to compute (start with 1, generate the next increment by multiplying by 3 and adding 1) and leads to a relatively efficient sort, even for moderately large files, as illustrated in Figure 6.10.

Many other increment sequences lead to a more efficient sort but it is difficult to beat the sequence in Program 6.5 by more than 20% even for relatively large *N*. One increment sequence that does so is 1 8 23 77 281 1073 4193 16577 ..., the sequence $4^{i+1} + 3 \cdot 2^i + 1$ for $i > 0$, which has provably faster worst-case behavior (*see Property 6.10*). Figure 6.12 shows that this sequence and Knuth's sequence—and many other sequences—have similar dynamic characteristics for large files.

The possibility that even better increment sequences exist is still real. A few ideas on improved increment sequences are explored in the exercises.

On the other hand, there are some bad increment sequences: for example 1 2 4 8 16 32 64 128 256 512 1024 2048 . . . (the original sequence suggested by Shell when he proposed the algorithm in 1959 (*see reference section*)) is likely to lead to bad performance because elements in odd positions are not compared against elements in even positions until the final pass. The effect is noticeable for random files, and is catastrophic in the worst case: The method degenerates to require quadratic running time if, for example, the half of the elements with the smallest values are in even positions and the half of the elements with the largest values are in the odd positions (See Exercise 6.36.)

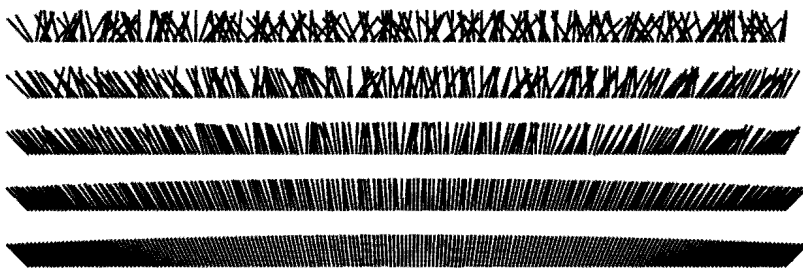
Program 6.5 computes the next increment by dividing the current one by 3, after initializing to ensure that the same sequence is always used. Another option is just to start with $h = N/3$ or with some other function of N . It is best to avoid such strategies, because bad sequences of the type described in the previous paragraph are likely to turn up for some values of N .

Our description of the efficiency of shellsort is necessarily imprecise, because no one has been able to analyze the algorithm. This gap in our knowledge makes it difficult not only to evaluate different increment sequences, but also to compare shellsort with other methods analytically. Not even the functional form of the running time for shellsort is known (furthermore, the form depends on the increment sequence). Knuth found that the functional forms $N(\log N)^2$ and $N^{1.25}$ both fit the data reasonably well, and later research suggests that a more complicated function of the form $N^{1+1/\sqrt{\lg N}}$ is involved for some sequences.

We conclude this section by digressing into a discussion of several facts about the analysis of shellsort that *are* known. Our primary purpose in doing so is to illustrate that even algorithms that are apparently simple can have complex properties, and that the analysis of algorithms is not just of practical importance but also can be intellectually challenging. Readers intrigued by the idea of finding a new and improved shellsort increment sequence may find the information that follows useful; other readers may wish to skip to Section 6.7.

Figure 6.10
Shellsorting a random permutation

The effect of each of the passes in Shellsort is to bring the file as a whole closer to sorted order. The file is first 40-sorted, then 13-sorted, then 4-sorted, then 1-sorted. Each pass brings the file closer to sorted order.



Property 6.7 *The result of h -sorting a file that is k -ordered is a file that is both h - and k -ordered.*

This fact seems obvious, but is tricky to prove (see Exercise 6.47). ■

Property 6.8 *Shellsort does less than $N(h-1)(k-1)/g$ comparisons to g -sort a file that is h - and k -ordered, provided that h and k are relatively prime.*

The basis for this fact is illustrated in Figure 6.11. No element farther than $(h-1)(k-1)$ positions to the left of any given element x can be greater than x , if h and k are relatively prime (see Exercise 6.43). When g -sorting, we examine at most one out of every g of those elements. ■

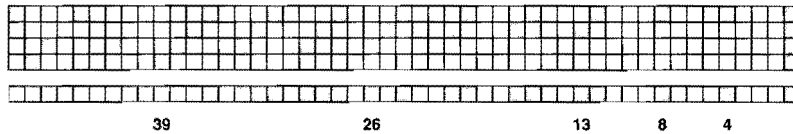
Property 6.9 *Shellsort does less than $O(N^{3/2})$ comparisons for the increments 1 4 13 40 121 364 1093 3280 9841*

For large increments, there are h subfiles of size about N/h , for a worst-case cost about N^2/h . For small increments, Property 6.8 implies that the cost is about Nh . The result follows if we use the better of these bounds for each increment. It holds for any relatively prime sequence that grows exponentially. ■

Property 6.10 *Shellsort does less than $O(N^{4/3})$ comparisons for the increments 1 8 23 77 281 1073 4193 16577*

The proof of this property is along the lines of the proof of Property 6.9. The property analogous to Property 6.8 implies that the cost for small increments is about $Nh^{1/2}$. Proof of this property requires number theory that is beyond the scope of this book (see reference section). ■

The increment sequences that we have discussed to this point are effective because successive elements are relatively prime. Another

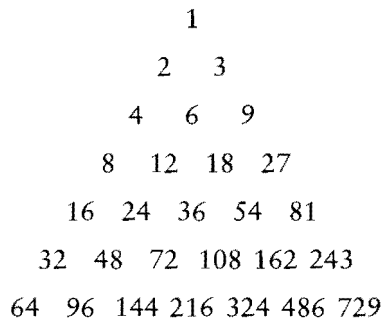


family of increment sequences is effective precisely because successive elements are *not* relatively prime.

In particular, the proof of Property 6.8 implies that, in a file that is 2-ordered and 3-ordered, each element moves at most one position during the final insertion sort. That is, such a file can be sorted with one bubble-sort pass (the extra loop in insertion sort is not needed). Now, if a file is 4-ordered and 6-ordered, then it also follows that each element moves at most one position when we are 2-sorting it (because each subfile is 2-ordered and 3-ordered); and if a file is 6-ordered and 9-ordered, each element moves at most one position when we are 3-sorting it. Continuing this line of reasoning, we are led to the following idea, which was developed by Pratt in 1971 (see reference section).

Property 6.11 *Shellsort does less than $O(N(\log N)^2)$ comparisons for the increments 1 2 3 4 6 9 8 12 18 27 16 24 36 54 81*

Consider the following triangle of increments, where each number in the triangle is two times the number above and to the right of it and also three times the number above and to the left of it.



If we use these numbers from bottom to top and right to left as a shellsort increment sequence, then every increment x after the bottom row is preceded by $2x$ and $3x$, so every subfile is 2-ordered and 3-ordered, and no element moves more than one position during the

Figure 6.11
A 4- and 13- ordered file.

The bottom row depicts an array, with shaded boxes depicting those items that must be smaller than or equal to the item at the far right, if the array is both 4- and 13-ordered. The four rows at top depict the origin of the pattern. If the item at right is at array position i , then 4-ordering means that items at array positions $i-4$, $i-8$, $i-12$, . . . are smaller or equal (top); 13-ordering means that the item at $i-13$, and, therefore, because of 4-ordering, the items at $i-17$, $i-21$, $i-25$, . . . are smaller or equal (second from top); also, the item at $i-26$, and, therefore, because of 4-ordering, the items at $i-30$, $i-34$, $i-38$, . . . are smaller or equal (third from top); and so forth. The white squares remaining are those that could be larger than the item at left; there are at most 18 such items (and the one that is farthest away is at $i-36$). Thus, at most $18N$ comparisons are required for an insertion sort of a 13-ordered and 4-ordered file of size N .

Table 6.2 Empirical study of shellsort increment sequences

Shellsort is many times faster than the other elementary methods even when the increments are powers of 2, but some increment sequences can speed it up by another factor of 5 or more. The three best sequences in this table are totally different in design. Shellsort is a practical method even for large files, particularly by contrast with selection sort, insertion sort, and bubble sort (see Table 6.1).

N	O	K	G	S	P	I
12500	16	6	6	5	6	6
25000	37	13	11	12	15	10
50000	102	31	30	27	38	26
100000	303	77	60	63	81	58
200000	817	178	137	139	180	126

Key:

O 1 2 4 8 16 32 64 128 256 512 1024 2048 ...

K 1 4 13 40 121 364 1093 3280 9841 ... (Property 6.9)

G 1 2 4 10 23 51 113 249 548 1207 2655 5843 ... (Exercise 6.40)

S 1 8 23 77 281 1073 4193 16577 ... (Property 6.10)

P 1 7 8 49 56 64 343 392 448 512 2401 2744 ... (Exercise 6.44)

I 1 5 19 41 109 209 505 929 2161 3905 ... (Exercise 6.45)

entire sort! The number of increments in the triangle that are less than N is certainly less than $(\log_2 N)^2$. ■

Pratt's increments tend not to work as well as the others in practice, because there are too many of them. We can use the same principle to build an increment sequence from *any* two relatively prime numbers h and k . Such sequences do well in practice because the worst-case bounds corresponding to Property 6.11 overestimate the cost for random files.

The problem of designing good increment sequences for shellsort provides an excellent example of the complex behavior of a simple algorithm. We certainly will not be able to focus at this level of detail on all the algorithms that we encounter (not only do we not have the space, but also, as we did with shellsort, we might encounter mathematical analysis beyond the scope of this book, or even open research prob-

lems). However, many of the algorithms in this book are the product of extensive analytic and empirical studies by many researchers over the past several decades, and we can benefit from this work. This research illustrates that the quest for improved performance can be both intellectually challenging and practically rewarding, even for simple algorithms. Table 6.2 gives empirical results that show that several approaches to designing increment sequences work well in practice; the relatively short sequence 1 8 23 77 281 1073 4193 16577 ... is among the simplest to use in a shellsort implementation.

Figure 6.13 shows that shellsort performs reasonably well on a variety of kinds of files, rather than just on random ones. Indeed, constructing a file for which shellsort runs slowly for a given increment sequence is a challenging exercise (see Exercise 6.42). As we have mentioned, there are some bad increment sequences for which shellsort may require a quadratic number of comparisons in the worst case (see Exercise 6.36), but much lower bounds have been shown to hold for a wide variety of sequences.

Shellsort is the method of choice for many sorting applications because it has acceptable running time even for moderately large files and requires a small amount of code that is easy to get working. In the next few chapters, we shall see methods that are more efficient, but they are perhaps only twice as fast (if that much) except for large N , and they are significantly more complicated. In short, if you need a quick solution to a sorting problem, and do not want to bother with interfacing to a system sort, you can *use shellsort*, then determine sometime later whether the extra work required to replace it with a more sophisticated method will be worthwhile.

Exercises

▷ 6.33 Is shellsort stable?

6.34 Show how to implement a shellsort with the increments 1 8 23 77 281 1073 4193 16577 ..., with direct calculations to get successive increments in a manner similar to the code given for Knuth's increments.

▷ 6.35 Give diagrams corresponding to Figures 6.8 and 6.9 for the keys E A S Y QUESTION.

6.36 Find the running time when you use shellsort with the increments 1 2 4 8 16 32 64 128 256 512 1024 2048 ... to sort a file consisting of the integers 1, 2, ..., N in the odd positions and $N+1, N+2, \dots, 2N$ in the even positions.

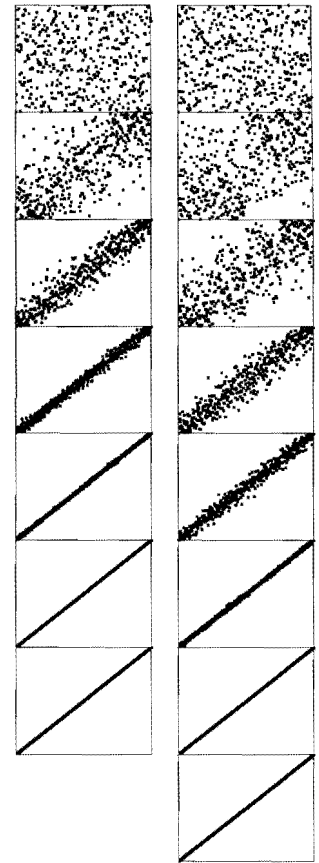
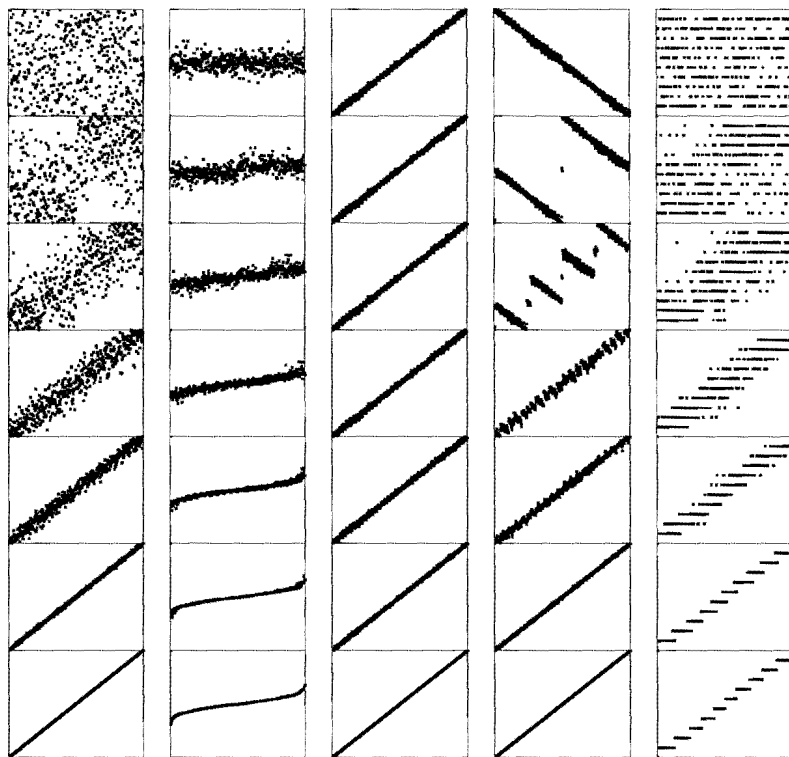


Figure 6.12
Dynamic characteristics of
shellsort (two different in-
crement sequences)

In this representation of shellsort in operation, it appears as though a rubber band, anchored at the corners, is pulling the points toward the diagonal. Two increment sequences are depicted: 121 40 13 4 1 (left) and 209 109 41 19 5 1 (right). The second requires one more pass than the first, but is faster because each pass is more efficient.

Figure 6.13
Dynamic characteristics of
shellsort for various types
of files

These diagrams show shellsort, with the increments 209 109 41 19 5 1, in operation on files that are random, Gaussian, nearly ordered, nearly reverse-ordered, and randomly ordered with 10 distinct key values (left to right, on the top). The running time for each pass depends on how well ordered the file is when the pass begins. After a few passes, these files are similarly ordered; thus, the running time is not particularly sensitive to the input.



6.37 Write a driver program to compare increment sequences for shellsort. Read the sequences from standard input, one per line, then use them all to sort 10 random files of size N for $N = 100$, 1000, and 10000. Count comparisons, or measure actual running times.

• **6.38** Run experiments to determine whether adding or deleting an increment can improve the increment sequence 1 8 23 77 281 1073 4193 16577 ... for $N = 10000$.

• **6.39** Run experiments to determine the value of x that leads to the lowest running time for random files when the 13 is replaced by x in the increment sequence 1 4 13 40 121 364 1093 3280 9841 ... used for $N = 10000$.

6.40 Run experiments to determine the value of α that leads to the lowest running time for random files for the increment sequence 1, $\lfloor \alpha \rfloor$, $\lfloor \alpha^2 \rfloor$, $\lfloor \alpha^3 \rfloor$, $\lfloor \alpha^4 \rfloor$, ...; for $N = 10000$.

• **6.41** Find the three-increment sequence that uses as small a number of comparisons as you can find for random files of 1000 elements.

- **6.42** Construct a file of 100 elements for which shellsort, with the increments 1 8 23 77, uses as large a number of comparisons as you can find.
- **6.43** Prove that any number greater than or equal to $(h-1)(k-1)$ can be expressed as a linear combination (with nonnegative coefficients) of h and k , if h and k are relatively prime. *Hint:* Show that, if any two of the first $h-1$ multiples of k have the same remainder when divided by h , then h and k must have a common factor.
- 6.44** Run experiments to determine the values of h and k that lead to the lowest running times for random files when a Pratt-like sequence based on h and k is used for sorting 10000 elements.
- 6.45** The increment sequence 1 5 19 41 109 209 505 929 2161 3905 ... is based on merging the sequences $9 \cdot 4^i - 9 \cdot 2^i + 1$ and $4^i - 3 \cdot 2^i + 1$ for $i > 0$. Compare the results of using these sequences individually and using the merged result, for sorting 10000 elements.
- 6.46** We derive the increment sequence 1 3 7 21 48 112 336 861 1968 4592 13776 ... by starting with a base sequence of relatively prime numbers, say 1 3 7 16 41 101, then building a triangle, as in Pratt's sequence, this time generating the i th row in the triangle by multiplying the first element in the $i-1$ st row by the i th element in the base sequence; and multiplying every element in the $i-1$ st row by the $i+1$ st element in the base sequence. Run experiments to find a base sequence that improves on the one given for sorting 10000 elements.
- **6.47** Complete the proofs of Properties 6.7 and 6.8.
- **6.48** Implement a shellsort that is based on the shaker sort algorithm of Exercise 6.30, and compare with the standard algorithm. *Note:* Your increment sequences should be substantially different from those for the standard algorithm.

6.7 Sorting of Other Types of Data

Although it is reasonable to learn most algorithms by thinking of them as simply sorting arrays of numbers into numerical order or characters into alphabetical order, it is also worthwhile to recognize that the algorithms are largely independent of the type of items being sorted, and that is not difficult to move to a more general setting. We have talked in detail about breaking our programs into independent modules to implement data types, and abstract data types (see Chapters 3 and 4); in this section, we consider ways in which we can apply the concepts discussed there to make our sorting implementations useful for various types of data.

Program 6.6 Sort driver for arrays

This driver for basic array sorts uses two explicit interfaces: one for the functions that initialize and print (and sort!) arrays, and the other for a data type that encapsulates the operations that we perform on generic items. The first allows us to compile the functions for arrays separately and perhaps to use them in other drivers; the second allows us to sort other types of data with the same sort code.

```
#include <stdlib.h>
#include "Item.h"
#include "Array.h"
main(int argc, char *argv[])
{ int i, N = atoi(argv[1]), sw = atoi(argv[2]);
  Item *a = malloc(N*sizeof(Item));
  if (sw) randinit(a, N); else scaninit(a, &N);
  sort(a, 0, N-1);
  show(a, 0, N-1);
}
```

Specifically, we consider implementations, interfaces, and client programs for:

- *Items*, or generic objects to be sorted
- *Arrays* of items

The item data type provides us with a way to use our sort code for any type of data for which certain basic operations are defined. The approach is effective both for simple data types and for abstract data types, and we shall consider numerous implementations. The array interface is less critical to our mission; we include it to give us an example of a multiple-module program that uses multiple data types. We consider just one (straightforward) implementation of the array interface.

Program 6.6 is a client program with the same general functionality of the main program in Program 6.1, but with the code for manipulating arrays and items encapsulated in separate modules, which gives us, in particular, the ability to test various sort programs on various different types of data, by substituting various different modules, but without changing the client program at all. To complete the imple-